

Este documento muestra la codificación directa para resolver los temas (salvo ciertos casos), si usted busca cómo aprender a resolver con las rutinas le recomiendo usar otro material sobre cómo funcionan las rutinas.

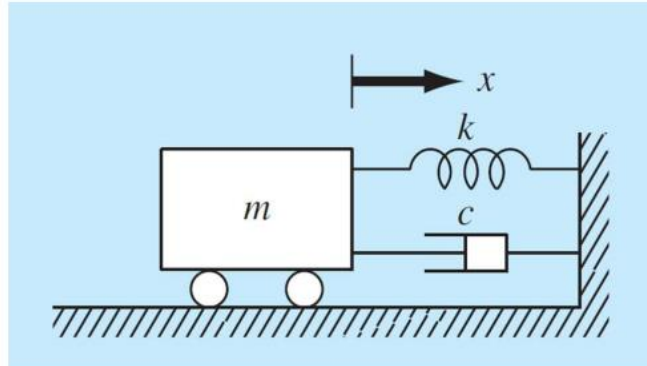
Formato del documento en la resolución de los ejercicios:

- **Año del tema y rutina que se usa para resolverlo empezando por 2021_02 hasta 2025_02.**
- **Enunciado con los ítems.**
- **Captura de pantalla del gráfico del ejercicio (si es que era necesario).**
- **Captura de una resolución a mano de cierto despeje o artificio (si es que era necesario).**
- **Código de resolución del problema donde puede hacer lectura o copiar y pegar en su programa para ejecutarlo.**
- **Captura de pantalla de las respuestas si solo necesita corroborar para no estar ejecutando los códigos de este documento en cada tema.**

Primer final 2021-02 Tema 2 (HAMMING MODIFICADO)

El movimiento de un sistema acoplado masa-resorte-amortiguador subamortiguado que se muestra en la figura está descrito por la ecuación diferencial ordinaria que sigue:

$$m \frac{d^2 x}{dt^2} + c \frac{dx}{dt} + kx = 0$$



Donde x es el desplazamiento desde la posición de equilibrio en metros, t es el tiempo en segundos, m es la masa del cuerpo de 23.6913 kg y c es el coeficiente de amortiguamiento. El coeficiente de amortiguamiento c es 4.4674 (N·s)/m. La constante del resorte es $k = 23.8072$ N/m. La velocidad inicial es de cero y el desplazamiento inicial es $x = 2.7023$ m. Utilizando el método de Hamming modificado con $h=0.1$ y el método de Runge Kutta O^4 para los cálculos auxiliares responda el cuestionario (utilizar 6 decimales para las respuestas; no utilizar notación científica; utilizar todos los decimales para los cálculos; no redondear las respuestas; utilizar el punto "." como separador decimal para las respuestas; NO ES NECESARIO HACER NINGUNA CONVERSION DE UNIDADES).

1. Analizando solo hasta $t = 6$ seg, el instante para cual el módulo de la aceleración alcanza su valor mínimo es:

✖

Una posible respuesta correcta sería: 0.0483

2. La fuerza sobre el resorte en valor absoluto para $t = 3$ seg es:

✖

Una posible respuesta correcta sería: 47.28358

3. La fuerza sobre el amortiguador en valor absoluto para $t = 4$ seg es:

✖

Una posible respuesta correcta sería: 6.265415

4. El módulo de la posición del cuerpo para $t = 7$ seg es:

✖

Una posible respuesta correcta sería: 1.150991

5. Analizando solo hasta $t = 10$ seg, el módulo de la velocidad máxima alcanzada es:

✖

Una posible respuesta correcta sería: 2.355692

```

#1F 2021_02 T2 (Hamming mod y Rk4)
import numpy as np
import os
os.system('cls')
np.set_printoptions(suppress=True)
#Datos
M=23.6913
c=4.4674
k=23.8072
h=0.1#paso
y0=np.array([[2.7023],[0]],dtype=float)#y0=[posición, velocidad]
x0=0#tiempo inicial
#Ec.diferencial: m*d2x/dt2+c*dx/dt+k*x=0

#Rk4 para puntos de hamming modificado
#####RUTINA023#####
m=len(y0)
a=np.hstack((np.zeros((m-1,1)),np.eye((m-1))))
A=lambda x:np.vstack((a,np.array([-k/M,-c/M]),dtype=float))
B=lambda x:np.vstack((np.zeros((m-1,1)),np.array([0]),dtype=float))
x=x0
R=[]
R.append([x,y0[0,0],y0[1,0],(A(x)@y0+B(x))[1,0]])
yn=[]
yn.append(y0)
n=3
for i in range(n):
    dy0=A(x)@y0+B(x)
    x+=h/2
    y1=y0+(h/2)*dy0
    dy1=A(x)@y1+B(x)
    y2=y0+(h/2)*dy1
    dy2=A(x)@y2+B(x)
    x+=h/2
    y3=y0+h*dy2
    dy3=A(x)@y3+B(x)
    y=y0+(h/6)*(dy0+2*dy1+2*dy2+dy3)
    y0=y.copy()
    R.append([x,y0[0,0],y0[1,0],(A(x)@y0+B(x))[1,0]])
    yn.append(y0)
#####RUTINA027#####Hamming modificado
y0,y1,y2,y3=yn
x1=x0+h
x2=x1+h
x3=x2+h
n=97
for i in range(n):
    dy1=A(x1)@y1+B(x1)
    dy2=A(x2)@y2+B(x2)

```

```

dy3=A(x3)*y3+B(x3)
p=y0+(4/3)*h*(2*dy1-dy2+2*dy3)
if i>0:
    mod=p+(112/121)*(y3-p0)
else:
    mod=p
x4=x3+h
dy4=A(x4)*mod+B(x4)
y=(9*y3-y1)/8+(3/8)*h*(-dy2+2*dy3+dy4)
y0=y1.copy()
y1=y2.copy()
y2=y3.copy()
y3=y.copy()
x1=x2
x2=x3
x3=x4
p0=p.copy()
R.append([x4,y[0,0],y[1,0],(A(x4)*y+B(x4))[1,0]])
Res=np.vstack(R)
print(Res)
a=np.abs(Res[:,3])
min=a[0]
for i in range(60):
    if a[i]<min:
        t=h*i
        min=a[i]
print('1-)instante para menor aceleración en el intervalo
[0,6][s]:',t)#Observación
#en la respuesta del ítem 1 da cla aceleración mínima pero te pide el
instante de esa aceleración
print('2-)La fuerza sobre el resorte en t=3[s]:',k*np.abs(Res[30,1]))
print('3-)La fuerza sobre el amortiguador en
t=4[s]:',c*np.abs(Res[40,2]))
print('4-)Posición del cuerpo (en módulo) para
t=7[s]:',np.abs(Res[70,1]))
v=np.abs(Res[:,2])
vmax=v[0]
for i in range(97):
    if vmax<v[i]:
        vmax=v[i]
print('5-)El módulo de la velocidad máxima:',vmax)

```

1-)instante para menor aceleración en el intervalo [0,6][s]: 1.5
2-)La fuerza sobre el resorte en t=3[s]: 47.28357987920434
3-)La fuerza sobre el amortiguador en t=4[s]: 6.265414561857929
4-)Posición del cuerpo (en módulo) para t=7[s]: 1.1509910283683802
5-)El módulo de la velocidad máxima: 2.355692120663781

Segundo final 2021-02 Tema 1 (MILNE MODIFICADO+RK04)

Teniendo en cuenta de que la función $y = f(x)$ cumple con la ecuación diferencial

$$(x+1)y'' - (3x+4)y' + 3y = (3x+2)e^{3x}$$

Se sabe que la curva pasa por el punto $(0; -3)$ y que la recta normal en el punto dado tiene por ecuación $x + y + 3 = 0$

Utilizando todos los decimales para los cálculos y un paso de $h = 0.025$; el método RK 04 para los valores iniciales, y el método de Milne modificado, determinar los valores solicitados (Utilizar 6 dígitos decimales, sin formato exponencial, y el punto como separador decimal):

a) (2 Ptos.) Estimar los valores de la ordenada para $x_1 = 0.1$

✖

Una posible respuesta correcta sería: -2.815155

b) (3 Ptos.) Determinar el error cometido en la estimación de $y_{(0.2)}$ teniendo que el valor real o exacto es de $\frac{6e^{0.6}-23}{5}$

$\times 10^{-7}$ (en este ítem tener en cuenta que ya se le da en formato exponencial, con lo que se debe multiplicar en el código por 10^7 para cargar la respuesta en este lugar)

✖

Una posible respuesta correcta sería: 1.292832

c) (2 Ptos.) Estimar los valores de la primera derivada para $x_3 = 0.15$

✖

Una posible respuesta correcta sería: 3.978989

refresh=304368&cid=76980

2do FINAL 2021 Cdo 2: Resolución del ítem

d) (3 Ptos.) Determinar el valor de $y''(0.2)$

✖

Una posible respuesta correcta sería: 30.611597

```

#2F 2021_02 (Milne mod y Rk4)
import numpy as np
import os
os.system('cls')
np.set_printoptions(suppress=True)
#Datos
h=0.025#paso
y0=np.array([-3],[1],dtype=float)#y0=[y, y'] (y'=-1/(pendiente
recta)= -1/-1 =-1)
x0=0#tiempo inicial
#Ec.diferencial: y"=(3*x+4)*y'/(x+1)-
3*y/(x+1)+(3*x+2)*np.exp(3*x)/(x+1)
#Solución real:
real=(6*np.exp(0.6)-23)/5
#Rk4 para puntos de hamming modificado
#####RUTINA023#####
m=len(y0)
a=np.hstack((np.zeros((m-1,1)),np.eye((m-1))))
A=lambda x:np.vstack((a,np.array([( -
3)/(x+1),(3*x+4)/(x+1)]),dtype=float)))
B=lambda x:np.vstack((np.zeros((m-
1,1)),np.array([(3*x+2)*np.exp(3*x)/(x+1)]),dtype=float)))
x=x0
R=[]
R.append([x,y0[0,0],y0[1,0],(A(x)@y0+B(x))[1,0]])
yn=[]
yn.append(y0)
n=3
for i in range(n):
    dy0=A(x)@y0+B(x)
    x+=h/2
    y1=y0+(h/2)*dy0
    dy1=A(x)@y1+B(x)
    y2=y0+(h/2)*dy1
    dy2=A(x)@y2+B(x)
    x+=h/2
    y3=y0+h*dy2
    dy3=A(x)@y3+B(x)
    y=y0+(h/6)*(dy0+2*dy1+2*dy2+dy3)
    y0=y.copy()
    R.append([x,y0[0,0],y0[1,0],(A(x)@y0+B(x))[1,0]])
    yn.append(y0)
#####RUTINA025#####
y0,y1,y2,y3=yn
x1=x0+h
x2=x1+h
x3=x2+h
n=17
for i in range(n):

```

```

dy1=A(x1)@y1+B(x1)
dy2=A(x2)@y2+B(x2)
dy3=A(x3)@y3+B(x3)
p=y0+(4/3)*h*(2*dy1-dy2+2*dy3)
if i>0:
    mod=p+(28/29)*(y3-p0)
else:
    mod=p
x4=x3+h
dy4=A(x4)@mod+B(x4)
y=y2+(h/3)*(dy2+4*dy3+dy4)
y0=y1.copy()
y1=y2.copy()
y2=y3.copy()
y3=y.copy()
x1=x2
x2=x3
x3=x4
p0=p.copy()
R.append([x4,y[0,0],y[1,0],(A(x4)@y+B(x4))[1,0]])
Res=np.vstack(R)
print(Res)
print('a-)Valor de la ordenada para x=0.1:',Res[int(round(0.1/h)),1])
print('b-)Error cometido en la estimación de y(0.2):',np.abs(real-
Res[int(round(0.2/h)),1]))
print('c-)Valor de d1y/dx en x=0.15:',Res[int(round(0.15/h)),2])
print('d-)Valor de d2y/dx2 en x=0.2:',Res[int(round(0.2/h)),3])

```

a-)Valor de la ordenada para x=0.1: -2.815155396255136

b-)Error cometido en la estimación de y(0.2): 1.2928327386418914e-07

c-)Valor de d1y/dx en x=0.15: 3.978989298333187

d-)Valor de d2y/dx2 en x=0.2: 30.6115974383438

Segundo final 2021-02 T2 (Simpson 1/3(033)+Falsa Posicion(010))

Las magnitudes de los campos eléctricos y magnéticos ejercidos por un imán sobre un cuerpo están definidas por las ecuaciones Graficar en el intervalo $[0, 10]$:

$$\text{Campo eléctrico } E = \frac{503+3 \times 9.27400915 \times 10^{-24}}{\sqrt{d^2+16}} \text{ [V/m]}$$

Campo magnético

$$B = e^{(-d+3)} + 200 - 15(d - 9.27400915 \times 10^{-24}) \text{ [T]}$$

$$\text{Fuerza del campo magnético } F = 0.72 \times B \text{ [N]}$$

Donde "d" es la distancia de separación entre el cuerpo y el imán. Si el cuerpo se encuentra inicialmente unido al imán y luego se aleja de forma unidireccional sobre una superficie plana debido a las fuerzas magnéticas, resuelva y responda el cuestionario utilizando los métodos de Simpson 1/3 con $s=11$ y la Falsa Posición con una tolerancia de 10^{-8} .

(utilizar 6 decimales para las respuestas; utilizar todos los decimales para los cálculos; no redondear las respuestas; utilizar el punto "." como separador decimal para las respuestas; NO ES NECESARIO HACER NINGUNA CONVERSION DE UNIDADES).

a) ¿A qué distancia del imán el Campo Eléctrico es máximo?

265.943330 ✖

Una posible respuesta correcta sería: 0

b) ¿Cuál es el valor del Campo Eléctrico en ese punto?

1.891166 ✖

Una posible respuesta correcta sería: 125.75

c) La distancia a la cual el campo eléctrico y magnético

idemp=334384icid=70800

3da FINAL 2021 Cero 2-Resolución del ítem 10
tienen la misma magnitud es:

10.298027 ✔

Una posible respuesta correcta sería: 10.298027

d) La distancia entre el imán y el cuerpo cuando la magnitud del campo magnético es de 150 T es:

-6.446549 ✖

Una posible respuesta correcta sería: 3.378971

e) Del ítem anterior, el trabajo realizado por la fuerza magnética hasta esa distancia es:

✖

Una posible respuesta correcta sería: 438.886366


```

#2F 2021_02 T2 (Simpson tercio y F_posición)
import numpy as np
import os
import matplotlib.pyplot as plt
import jax.numpy as jnp
from jax import grad
os.system('cls')
np.set_printoptions(suppress=True)
#Datos
s=11#cantidad de polinomios para simpson 1/3
tol=1e-8
def E(d):
    return (503+3*9.27400915*1e-24)/((d**2+16)**0.5)
def B(d):
    return np.exp(-d+3)+200-15*(d-9.27400915*1e-24)
def F(b):
    return 0.72*b

#falsa posición
def f_pos(f,a,b,n):
    #####RUTINA005##### falsa posición
    c0=a
    for i in range(n):
        c=(b*f(a)-a*f(b))/(f(a)-f(b))
        err=np.abs(c0-c)
        relerr=np.abs(err/c)
        if tol>err or tol>relerr or tol>np.abs(f(c)):
            break
        else:
            if f(a)*f(c)<0:
                b=c
            else:
                a=c
            c0=c
    return c

#simpson 1/3
def simp(f,a,b,n):
    #####RUTINA015#####
    h=(b-a)/n
    A=0
    y=f(np.linspace(a,b,n+1))
    k=0
    while(k<n):
        A+=(h/3)*(y[k]+4*y[k+1]+y[k+2])
        k+=2
    return A

#graficación

```

```

def graficar(f,a,b,m,nombre):
    xvals=np.linspace(a,b,m)
    plt.plot(xvals,f(xvals),label=nombre)
    plt.legend()
    plt.grid(True)
    return
graficar(E,0,10,50,"Campo Eléctrico")
plt.show()
dEmax=f_pos(grad(E),-1.0,0.0,100)#distancia para máximo de la función
campo eléctrico
print('a-)Distancia para campo eléctrico máximo:',dEmax)
print('b-)Valor máximo del campo eléctrico:',E(dEmax))
EB=lambda x:E(x)-B(x)
graficar(EB,0,10,50,"(E-B)(d)")
plt.show()
print('c-)Distancia para que E=B:',f_pos(EB,0,1,100))
d150=f_pos(lambda x:B(x)-150,0,10,100)
print('d-)Distancia entre el imán y el cuerpo:',d150)
print('e-)Trabajo realizado por la fuerza magnética hasta esa
distancia:',simp(lambda x:F(B(x)),0,d150,2*s))

```

a-)Distancia para campo eléctrico máximo: 0.0

b-)Valor máximo del campo eléctrico: 125.75

c-)Distancia para que E=B: 10.298027109389764

d-)Distancia entre el imán y el cuerpo: 3.3789710365301837

e-)Trabajo realizado por la fuerza magnética hasta esa distancia: 438.88636735736895

Segundo final 2021-02 Tema3 (Trapecios + Int. Newton)

Teniendo en cuenta que:

$$\int_1^3 e^x (x^3 + 3) dx = 15e^3 - e \quad (*)$$

Resuelva y responda el cuestionario

I) utilizando primero el polinomio interpolador que aproxima a la función que se quiere integrar tomando las abscisas $x=1, x=1.5, x=2, x=2.5, x=3$.

II) Utilizando el método del Trapecio tomando las abscisas $x=1, x=1.5, x=2, x=2.5, x=3$.

(utilizar 6 decimales para las respuestas; utilizar todos los decimales para los cálculos; no redondear las respuestas; utilizar el punto "." como separador decimal para las respuestas; NO ES NECESARIO HACER NINGUNA CONVERSION DE UNIDADES).

a) Evaluar el polinomio del método (I) para $x=2.3$

150.500101 ☒

Una posible respuesta correcta sería: 150.500101

b) ¿Cuál es el error absoluto cometido al usar el método (I), (utilizar el polinomio para hallar ALGEBRAICAMENTE el valor de la integral definida) para aproximar el valor de la integral (*)?

☒

Una posible respuesta correcta sería: 0.201028

c) Cual es el valor aproximado de la integral al utilizar el método (II, utilice el método del trapecio):

313.458572 ☒

Una posible respuesta correcta sería: 321.734476

d) Cual es el error absoluto cometido al utilizar el método (II), (determine el valor aproximado de la integral por el

Tidurpqr=0043046.comid=76880

2do FINAL 2021 Q de 2: Resolución del ítem
método del trapecio):

14.951693 ☒

Una posible respuesta correcta sería: 23.169704

e) Cual de los métodos es el que posee menor error:

☒ a. I

☐ b. II

☒

Una posible respuesta correcta sería: I

#2F 2021_02 T3 (Trapecios y Int_newton)

```
import numpy as np
import os
import matplotlib.pyplot as plt
import jax.numpy as jnp
from jax import grad
os.system('cls')
np.set_printoptions(suppress=True)
def fx(x):
```

```

    return np.exp(x)*(x**3+3)
xvals=np.linspace(1,3,5)
yvals=fx(xvals)
#interpolación de newton
def int_newton(x,y):
#####RUTINA013#####
    n=len(x)
    p=np.poly1d([1,-x[0]])
    P=np.poly1d(y[0])
    for i in range(1,n):
        a=(y[i]-P(x[i]))/p(x[i])
        P+=a*p
        p=np.polymul(p,np.poly1d([1,-x[i]]))
    return P
#Método del trapecio
def trapecio(f,a,b,n):
#####RUTINA014#####
    h=(b-a)/n
    A=0
    y=f(np.linspace(a,b,n+1))
    k=0
    while(k<n):
        A+=(h/2)*(y[k]+y[k+1])
        k+=1
    return A
#Polinomio interpolador
F=int_newton(xvals,yvals)
print(F)
print('a-)x=2.3 en polinomio interpolador:',F(2.3))
print('b-)Error absoluto para la integración:',(F.integ()(3)-
F.integ()(1))-(15*np.exp(3)-np.exp(1)))
print('c-)Valor aproximado de la intergral:',trapecio(fx,1,3,4))
print('d-)Error absoluto al integrar con trapecio:',trapecio(fx,1,3,4)-
(15*np.exp(3)-np.exp(1)))
print('e-)Método que posee menos error:',1,'(con polinomio
interpolador)')

```

```

a-)x=2.3 en polinomio interpolador: 150.50010165933577
b-)Error absoluto para la integración: 0.20102844849515122
c-)Valor aproximado de la intergral: 321.7344762172388
d-)Error absoluto al integrar con trapecio: 23.169704197882822
e-)Método que posee menos error: 1 (con polinomio interpolador)

```

Primer final 2022-01 Tema 1 (HAMMING MODIFICADO y Rk4)

TEMA 1

Utilizando el método de HAMMING - Modificado, responda el siguiente cuestionario:

Para los cálculos utilizar todos los decimales, para las respuestas utilizar DIEZ cifras decimales sin redondear el último decimal. Para el cálculo de los valores faltantes utilizar RKO⁴.

Dada la ecuación diferencial

$$y' = \frac{3x}{1+y}$$

bajo la condición inicial de

$$y(0) = 1, h = 0.1$$

Calcular y responder los siguientes valores solicitados:

1. Valor de la ordenada en $x=0.1$: ✖ @1,0074859987o1.0074859987@
2. Valor de la ordenada en $x=0.2$: ✖ @1,0297783463o1.0297783463@
3. Valor de la ordenada en $x=0.3$: ✖ @1,0663979009o1.0663979009@
4. Valor de la ordenada en $x=0.4$: ✖ @1,1166031679o1.1166031679@
5. Valor de la ordenada en $x=0.5$: ✖ @1,1794537769o1.1794537769@

```
#1F 2022_01 T1 (Hamming mod y RK4)
import numpy as np
import os
os.system('cls')
np.set_printoptions(suppress=True)
#Datos
h=0.1
y0=np.array([1],dtype=float)
x0=0
#ec. diferencia;: y'=3*x/(1+y)

#Rk4 para puntos faltantes en hamming
#####RUTINA023#####
m=len(y0)
a=np.hstack((np.zeros((m-1,1)),np.eye((m-1))))
A=lambda x:np.vstack((a,np.array([[0]],dtype=float)))
B=lambda x,y:np.vstack((np.zeros((m-1,1)),np.array([[3*x/(1+y)],dtype=float)))
x=x0
yn=[]
yn.append(y0)
R=[]
R.append([x,y0[0],(A(x)@y0+B(x,y0[0]))[0][0]])
n=3
for i in range(n):
    dy0=A(x)@y0+B(x,y0[0])
    x+=h/2
```

```

y1=y0+(h/2)*dy0
dy1=A(x)@y1+B(x,y1[0])
y2=y0+(h/2)*dy1
dy2=A(x)@y2+B(x,y2[0])
x+=h/2
y3=y0+h*dy2
dy3=A(x)@y3+B(x,y3[0])
y=y0+(h/6)*(dy0+2*dy1+2*dy2+dy3)
y0=y.copy()
yn.append(y0)
R.append([x,y[0][0],(A(x)@y0+B(x,y[0]))[0][0]])

#####ROUTINA027#####
y0,y1,y2,y3=yn
x1=x0+h
x2=x1+h
x3=x2+h
2
for i in range(n):
    dy1=A(x1)@y1+B(x1,y1[0])
    dy2=A(x2)@y2+B(x2,y2[0])
    dy3=A(x3)@y3+B(x3,y3[0])
    p=y0+(4/3)*h*(2*dy1-dy2+2*dy3)
    if i>0:
        mod=p+(112/121)*(y3-p0)
    else:
        mod=p
    x4=x3+h
    dy4=A(x4)@mod+B(x4,mod[0])
    y=(9*y3-y1)/8+(3/8)*h*(-dy2+2*dy3+dy4)
    y0=y1.copy()
    y1=y2.copy()
    y2=y3.copy()
    y3=y.copy()
    x1=x2
    x2=x3
    x3=x4
    p0=p.copy()
    R.append([x4,y[0][0],(A(x4)@y+B(x4,y[0]))[0][0]])
Res=np.vstack(R)
print('1-)Valor de la ordenada en x=0.1',Res[1,1])
print('2-)Valor de la ordenada en x=0.2',Res[2,1])
print('3-)Valor de la ordenada en x=0.3',Res[3,1])
print('4-)Valor de la ordenada en x=0.4',Res[4,1])
print('5-)Valor de la ordenada en x=0.5',Res[5,1])

```

```

1-)Valor de la ordenada en x=0.1 1.0074859987120173
2-)Valor de la ordenada en x=0.2 1.0297783463069556
3-)Valor de la ordenada en x=0.3 1.0663979009931648
4-)Valor de la ordenada en x=0.4 1.1166031679002157
5-)Valor de la ordenada en x=0.5 1.179453776958304

```

Segundo final 2022-01 Tema 2 (RKO4)

Un cuerpo de peso $W=8$ [N] y volumen $V=0.001$ [m³] que se encuentra completamente sumergido en un líquido de densidad $\rho = 1000$ kg/m³ y viscosidad $c = 1.003$ Ns/m, se le imprime una velocidad para abajo de $v_0 = 10$ [m/s] en $t=0$ [s], considerando el sistema de referencia desde la superficie y para abajo. Utilizar $g = 10$ m/s²

Considerar la ecuación diferencial del movimiento del cuerpo

$$W - \rho g V - c \frac{dy}{dt} = \frac{W}{g} \frac{d^2 y}{dt^2}$$

Utilizando el método de RKO4 con un paso de $h = 0.1$ s (utilizar 6 decimales para las respuestas sin realizar ningún redondeo; no utilizar notación exponencial; utilizar todos los decimales para los cálculos, utilizar el "punto" como separador decimal. Utilizar el signo según el sistema de referencia dado, no es necesario realizar ninguna conversión de unidades).

Determinar, utilizando los instantes tabulados con paso h (en el instante inmediatamente anterior a lo solicitado):

- a) La aceleración en el punto de máxima profundidad

 ✖

Una posible respuesta correcta sería: -2.599453

- b) La velocidad al llegar a la superficie

 ✖

Una posible respuesta correcta sería: -1.960917

- c) La profundidad máxima alcanzada por el cuerpo

 ✖

Una posible respuesta correcta sería: 5.121176

- d) El instante en que vuelve a la superficie

 ✖

Una posible respuesta correcta sería: 4.7

- e) El instante en el que el cuerpo se detiene

 ✖

Una posible respuesta correcta sería: 1.4

```
#2F 2022_01 T2 ( RK4 )
import numpy as np
import os
os.system('cls')
np.set_printoptions(suppress=True)
#Datos
w=8
v=0.001
y0=np.array([[0],[10]],dtype=float)#Valores iniciales y(0)=[y(0),y'(0)]
(posición y velocidad)
x0=0#tiempo inicial
p=1000
c=1.003
g=10
h=0.1
#Ec. diferencial despejada:
#d2y=0*y-c*g/w*y'+g-p*g**2*v/w
'''-----
```

No es necesario realizar modificaciones a la rutina

```
-----'''  
#####RUTINA023##### Rk4  
m=len(y0)  
a=np.hstack((np.zeros((m-1,1)),np.eye((m-1))))  
A=lambda x:np.vstack((a,np.array([0,-c*g/w]),dtype=float))  
B=lambda x:np.vstack((np.zeros((m-1,1)),np.array([g-  
p*g**2*v/w]),dtype=float))  
x=x0  
R=[]  
R.append((x0,y0[0][0],y0[1][0],(A(x0)@y0+B(x0)).flatten()[1]))#Iniciali  
zación matriz de cálculos  
n=120  
for i in range(n):  
    dy0=A(x)@y0+B(x)  
    x+=h/2  
    y1=y0+(h/2)*dy0  
    dy1=A(x)@y1+B(x)  
  
    y2=y0+(h/2)*dy1  
    dy2=A(x)@y2+B(x)  
  
    x+=h/2  
    y3=y0+h*dy2  
    dy3=A(x)@y3+B(x)  
  
    y=y0+(h/6)*(dy0+2*dy1+2*dy2+dy3)  
    y0=y.copy()  
    R.append((x,y0[0][0],y0[1][0],(A(x0)@y0+B(x0)).flatten()[1]))  
  
Res=np.vstack(R)  
print(Res)#Se imprime matriz de resultados para sacar respuestas  
print('a-)Aceleración en el punto de máxima profundidad:',Res[14,3])  
print('b-)Velocidad al llegar a la superficie',Res[47,2])  
p=Res[:,1]  
pmax=p[0]  
tpmax=0#tiempo al que corresponde la profundidad máxima (también cuando  
el cuerpo se detiene)  
for i in range(len(p)):  
    if pmax<p[i]:  
        pmax=p[i]  
        tpmax=i*h  
print('c-)Profundidad máxima alcanzada por el cuerpo',pmax)  
print('d-)Instante en que vuelve a la superficie:',4.7)  
print('f-)El instante en que el cuerpo se detiene',tpmax)
```

a-)Aceleración en el punto de máxima profundidad: -2.5994531450105516

b-)Velocidad al llegar a la superficie -1.9609173341692199

c-)Profundidad máxima alcanzada por el cuerpo 5.121176835588198

d-)Instante en que vuelve a la superficie: 4.7

f-)El instante en que el cuerpo se detiene 1.4000000000000001

Primer final 2022-01 Tema 3 (lagrange, Falsa pos. Y Simpson 1/3)

Una máquina opera diariamente según los datos de la tabla, que muestra la temperatura de la misma en función del tiempo. Si la máquina se enfría, o se calienta demás, unos reguladores de temperatura se accionan hasta que la temperatura llegue a cero, para que la misma se mantenga en el rango de temperatura operacional.

Por otro lado, debido al aumento de temperatura la máquina se dilata horizontalmente, produciéndose una fuerza de compresión sobre la misma según la ecuación:

$F = x^2 + e^x + 5x - 1$ donde F : fuerza horizontal [kN] ; x : deformación horizontal de la máquina [mm]

Tiempo (h)	Temperatura(°C)	Deformación(mm)
1.2	-0.450000	0.60000
1.82	0.505041	0.327464
3.24	0.690000	1.320000
4.68	-0.630000	1.20000
6.26	-0.550000	0.840000
6.88	0.660000	0.730000

Utilizando los métodos de Lagrange, falsa posición con $\text{tol} = 10^{-8}$ y Simpson 1/3 con $s=7$ responda el cuestionario (utilizar seis decimales para las respuestas sin redondear el último decimal; utilizar todos los decimales para los cálculos; no utilizar formato exponencial; No es necesario realizar ninguna conversión de unidades):

1. El primer instante en el cual el regulador se desactiva es:

h



Una posible respuesta correcta sería: 1.448707

2. La fuerza sobre la máquina en el instante 2.875372 hs es:

kN



Una posible respuesta correcta sería: 8.868633

3. El trabajo efectuado sobre la máquina en los instantes 1.6 a 2.678353 hs es:

J

✖

Una posible respuesta correcta sería: 3.037335

4. La deformación de la maquina en el instante 5.001151 hs es:

mm

✔

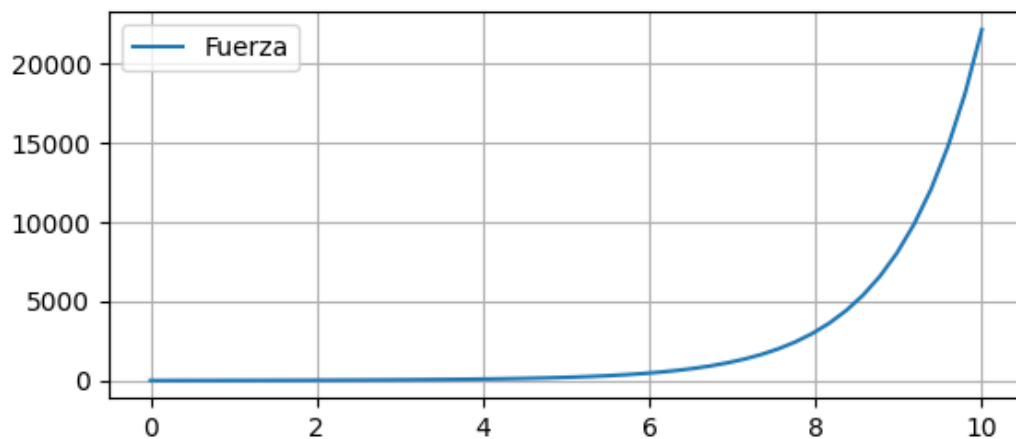
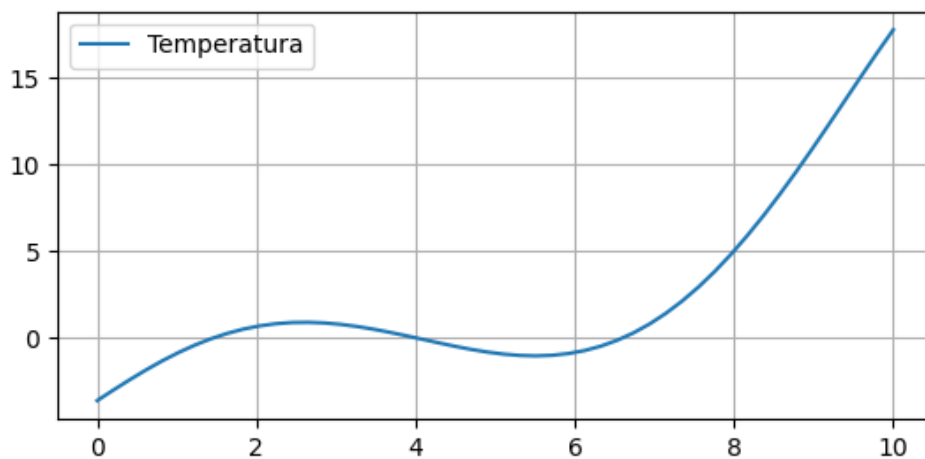
Una posible respuesta correcta sería: 1.076473

5. Si se aplica una fuerza de 3.284092 kN sobre la máquina, entonces la deformación menor es de:

mm

✖

Una posible respuesta correcta sería: 0.48496



```
#1F 2022_01 T3 (Lagrange, F_pos y simpson tercio)
import numpy as np
import matplotlib.pyplot as plt
import os
os.system('cls')
np.set_printoptions(suppress=True)
#Datos
tiempo=np.array([1.2,1.82,3.24,4.68,6.26,6.88],dtype=float)
```

```

temperatura=np.array([-0.45,0.505041,0.69,-0.63,-
0.55,0.66],dtype=float)
deformación=np.array([0.6,0.327464,1.32,1.2,0.84,0.73],dtype=float)
s=7#cantidad de polinomios para integrar
tol=1e-8
def F(x):#Fuerza debido a la deformación "x"
    return x**2+np.exp(x)+5*x-1
def lag(x,y):#Interpolación de lagrange
    #####RUTINA012#####
    n=len(x)
    P=0
    for i in range(n):
        a=np.delete(np.arange(n),i)
        p=np.poly1d([1,-x[a[0]]])
        for j in range(1,n-1):
            p=np.polymul(p,np.poly1d([1,-x[a[j]]]))
        P+=y[i]*p/p(x[i])
    return P

def f_pos(f,a,b,n):#falsa posición
    #####RUTINA005#####
    c0=a
    for i in range(n):
        c=(b*f(a)-a*f(b))/(f(a)-f(b))
        err=np.abs(c0-c)
        relerr=np.abs(err/c)
        if tol>err or tol>relerr or tol>np.abs(f(c)):
            break
        else:
            if f(a)*f(c)<0:
                b=c
            else:
                a=c
            c0=c
    return c

def simp(f,a,b,n):#simpson 1/3
    #####RUTINA015#####
    h=(b-a)/n
    A=0
    y=f(np.linspace(a,b,n+1))
    k=0
    while(k<n):
        A+=(h/3)*(y[k]+4*y[k+1]+y[k+2])
        k+=2
    return A

def graficar(f,a,b,m,nombre):
    xvals=np.linspace(a,b,m)

```

```

plt.plot(xvals,f(xvals),label=nombre)
plt.legend()
plt.grid(True)
return
#función temperatura
temp=np.poly1d(lag(tiempo,temperatura))
#función deformación
defor=np.poly1d(lag(tiempo,deformación))
#gráfico de función temperatura
graficar(temp,0,10,50,"Temperatura")
plt.show()
t1=f_pos(temp,1,2,100)
print('1-)1er instante en el que el regulador se desactiva:',t1)
print('2-)Fuerza sobre la máquina en
t=2.875372[hs]:',F(defor(2.875372)))
trabajo=simp(lambda x:F(x),defor(1.6),defor(2.678353),2*s)
print('3-)El trabajo efectuado sobre la máquina en
[1.6,2.678353]:',trabajo)
print('4-)Deformación de la máquina en t=5.001151:',defor(5.001151))
graficar(lambda x:F(x),0,10,50,"Fuerza")
plt.show()
print('5-)Deformación para una fuerza de 3.284092[kN]:',f_pos(lambda
x:F(x)-3.284092,0.4,0.5,50))

```

1-)1er instante en el que el regulador se desactiva: 1.4487066864965186
2-)Fuerza sobre la máquina en t=2.875372[hs]: 8.868636669193283
3-)El trabajo efectuado sobre la máquina en [1.6,2.678353]: 3.037334613924757
4-)Deformación de la máquina en t=5.001151: 1.07647323716634
5-)Deformación para una fuerza de 3.284092[kN]: 0.4849594410278825

OBSERVACIÓN: EL ENUNCIADO DEBERÍA DE DAR DE DATO EL VOLUMEN DE LA BOYA CILINDRICA, VOLUMEN= $2m^3$.

Una boya cilíndrica de $1500 [kg]$ y área de base $1 [m^2]$, flota verticalmente en agua con $1/4$ de su volumen emergiendo del líquido. En ese momento se sube una masa de $200 [kg]$ y la boya comienza a oscilar, suponiendo que la fuerza viscosa de magnitud igual a $1/4$ de la velocidad instantánea del movimiento. Considerar, gravedad $10 [m/s^2]$, densidad del agua $1000 [kg/m^3]$. Además colocar el sistema de referencia en la superficie del líquido y con el sentido positivo para abajo, considerar $t = 0 [s]$ cuando se sube la masa a la boya. Para las ecuaciones considera a y como la posición de la base inferior de la boya (en metros).

Utilizando un paso de $h = 0.1 [s]$ para el método de Hamming Modificado, y RKO4 para los valores iniciales. Armar una tabla de $y = y(t)$.

Responder el cuestionario utilizando seis decimales para las respuestas sin redondear el último decimal; utilizar todos los decimales para los cálculos; no utilizar formato exponencial (cargar los resultados con su signo):

Calcular:

1) la distancia adicional aproximada que la boya se hundirá hasta un poco antes de detenerse por primera vez.

✖

Una posible respuesta correcta sería: 0.394701

2) el instante justo antes que se detenga en el que sucede el ítem a)

✖

Una posible respuesta correcta sería: 1.2

3) La velocidad en $t=0.2 s$

✖

Una posible respuesta correcta sería: 0.226165

4) La aceleración en el instante $t=0.4 s$

✖

Una posible respuesta correcta sería: 0.665027

5) La posición de la base inferior de la boya en $t=0.6$ s

✖

Una posible respuesta correcta sería: 1.676935

6) La fuerza neta sobre el sistema boya+masa en $t=0.8$ s

✖

Una posible respuesta correcta sería: -637.378309

7) La velocidad en $t=1.0$ s

✖

Una posible respuesta correcta sería: 0.318432

8) La aceleración en el instante $t=1.2$ s

✖

Una posible respuesta correcta sería: -1.145341

9) La posición de la base inferior de la boya en $t=1.4$ s

✖

Una posible respuesta correcta sería: 1.893592

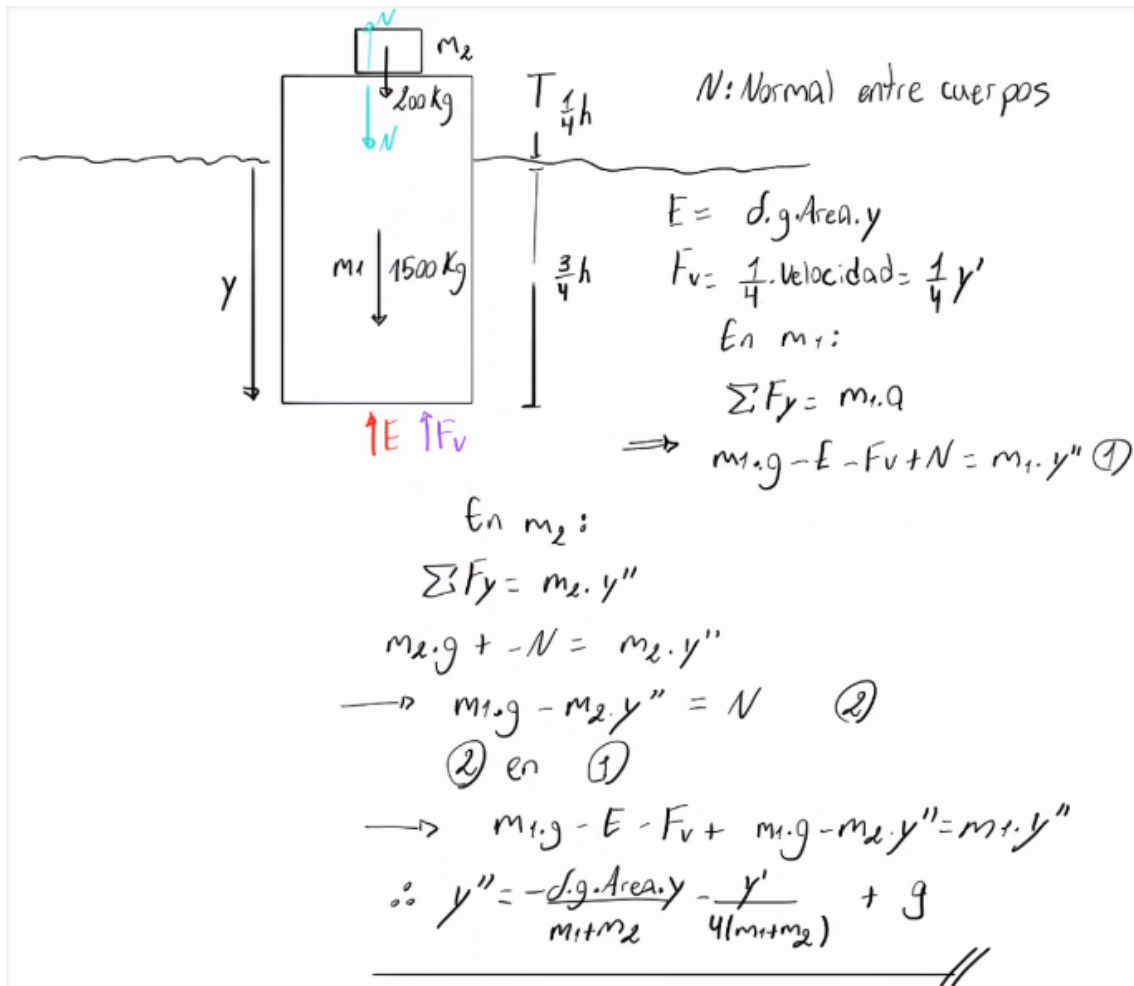
10) La fuerza neta sobre el sistema boya+masa en $t=1.6$ s

✖

Una posible respuesta correcta sería: -1708.248854

Faltó escribir en la captura que ambos cuerpos tienen la misma aceleración porque nunca se separan por eso en cada sumatoria se puede poner la misma " y'' ".

Otra forma de llegar a la E.D es tomar como dcl el sistema completo de las dos masas, esta captura se resuelve mediante cada dcl por separado.



N : Normal entre cuerpos

$E = \rho \cdot g \cdot \text{Area} \cdot y$

$F_v = \frac{1}{4} \cdot \text{velocidad} = \frac{1}{4} y'$

En m_1 :

$$\sum F_y = m_1 \cdot a$$

$$\Rightarrow m_1 \cdot g - E - F_v + N = m_1 \cdot y'' \quad (1)$$

En m_2 :

$$\sum F_y = m_2 \cdot y''$$

$$m_2 \cdot g + (-N) = m_2 \cdot y''$$

$$\rightarrow m_1 \cdot g - m_2 \cdot y'' = N \quad (2)$$

(2) en (1)

$$\rightarrow m_1 \cdot g - E - F_v + m_1 \cdot g - m_2 \cdot y'' = m_1 \cdot y''$$

$$\therefore y'' = \frac{-\rho \cdot g \cdot \text{Area} \cdot y}{m_1 + m_2} - \frac{y'}{4(m_1 + m_2)} + g$$

```
#1F 2022_02 T1 ( Hamming Mod y RK4 )
import numpy as np
import os
os.system('cls')
np.set_printoptions(suppress=True)
#Datos
M=1500 #Masa del cilindro
Ab=1 #area de base del cilindro
g=10 #gravedad
po=1000 #densidad del agua
h=0.1
#Ec. diferencial: y''=-po*g*Ab*y/(200+M)-y'/(4*(M+200))+g
y0=np.array([[1.5],[0]],dtype=float)#y0=[posición, velocidad]
```

```

x0=0

#Rk4 para puntos necesarios en hamming mod

#####RUTINA023##### RK4
m=len(y0)
a=np.hstack((np.zeros((m-1,1)),np.eye((m-1))))
A=lambda x:np.vstack((a,np.array((-po*Ab*g/(200+M), -
1/(4*(M+200))))),dtype=float))
B=lambda x:np.vstack((np.zeros((m-1,1)),np.array([g]),dtype=float))
x=x0
yn=[]
yn.append(y0)
R=[]
R.append([x,y0[0,0],y0[1,0],(A(x)@y0+B(x))[1,0]])
n=3
for i in range(n):
    dy0=A(x)@y0+B(x)
    x+=h/2
    y1=y0+(h/2)*dy0
    dy1=A(x)@y1+B(x)
    y2=y0+(h/2)*dy1
    dy2=A(x)@y2+B(x)
    x+=h/2
    y3=y0+h*dy2
    dy3=A(x)@y3+B(x)
    y=y0+(h/6)*(dy0+2*dy1+2*dy2+dy3)
    y0=y.copy()
    yn.append(y0)
    R.append([x,y0[0,0],y0[1,0],(A(x)@y0+B(x))[1,0]])

#####RUTINA027##### Hamming Modificado
y0,y1,y2,y3=yn
x1=x0+h
x2=x1+h
x3=x2+h
n=20
for i in range(n):
    dy1=A(x1)@y1+B(x1)
    dy2=A(x2)@y2+B(x2)
    dy3=A(x3)@y3+B(x3)
    p=y0+(4/3)*h*(2*dy1-dy2+2*dy3)
    if i>0:
        mod=p+(112/121)*(y3-p0)
    else:
        mod=p
    x4=x3+h
    dy4=A(x4)@mod+B(x4)
    y=(9*y3-y1)/8+(3/8)*h*(-dy2+2*dy3+dy4)

```



```

y0=y1.copy()
y1=y2.copy()
y2=y3.copy()
y3=y.copy()
x1=x2
x2=x3
x3=x4
p0=p.copy()
R.append([x4,y[0,0],y[1,0],(A(x4)*y+B(x4))[1,0]])
Res=np.vstack(R)
print(Res)
print('1-)Distancia adicional que la boya de hundirá hasta detenerse
por primera vez:',Res[12,1]-1.5)
print('2-)Instante justo antes de que se detenga:',1.2)
print('3-)Velocidad en t=0.2[s]:',Res[2,2])
print('4-)Aceleración en t=0.4[s]:',Res[4,3])
print('5-)Posición en t=0.6[s]:',Res[6,1])
print('6-)Fuerza neta sobre el sistema boya+masa en
t=0.8[s]:',(M+200)*Res[8,3])
print('7-)Velocidad en t=1.0[s]:',Res[10,2])
print('8-)Aceleración en t=1.2[s]:',Res[12,3])
print('9-)Posición de la boya en t=1.4[s]:',Res[14,1])
print('10-)Fuerza neta sobre el sistema boya+masa en
t=1.6[s]:',(M+200)*Res[14,3])

```

```

1-)Distancia adicional que la boya de hundirá hasta detenerse por primera vez: 0.39470109
2-)Instante justo antes de que se detenga: 1.2
3-)Velocidad en t=0.2[s]: 0.22616500984981783
4-)Aceleración en t=0.4[s]: 0.6648960274482203
5-)Posición en t=0.6[s]: 1.6769357552625042
6-)Fuerza neta sobre el sistema boya+masa en t=0.8[s]: -722.5210143962985
7-)Velocidad en t=1.0[s]: 0.3184329904581841
8-)Aceleración en t=1.2[s]: -1.1453168582116078
9-)Posición de la boya en t=1.4[s]: 1.8935921674232894
10-)Fuerza neta sobre el sistema boya+masa en t=1.6[s]: -1935.8911786069375

```

2do final 2022_02 Tema2 (Rk4)

Un cuerpo de peso $W=8 \text{ [N]}$ y volumen $V=0.001 \text{ [m}^3\text{]}$ que se encuentra completamente sumergido en un líquido de densidad $\rho = 1000 \text{ kg/m}^3$ y viscosidad $c = 1.003 \text{ N s/m}$, se le imprime una velocidad para abajo de $v_o = 10 \text{ [m/s]}$ en $t=0 \text{ [s]}$, considerando el sistema de referencia desde la superficie y para abajo. Utilizar $g = 10 \text{ m/s}^2$

Considerar la ecuación diferencial del movimiento del cuerpo

$$W - \rho g V - c \frac{dy}{dt} = \frac{W}{g} \frac{d^2 y}{dt^2}$$

Utilizando el método de RK04 con un paso de $h = 0.1 \text{ s}$ (utilizar 6 decimales para las respuestas sin realizar ningún redondeo; no utilizar notación exponencial; utilizar todos los decimales para los cálculos, utilizar el "punto" como separador decimal. Utilizar el signo según el sistema de referencia dado, no es necesario realizar ninguna conversión de unidades).

Determinar, utilizando los instantes tabulados con paso h (en el instante inmediatamente anterior a lo solicitado):

- a) La aceleración en el punto de máxima profundidad

 ✖

Una posible respuesta correcta sería: -2.599453

- b) La velocidad al llegar a la superficie

 ✖

Una posible respuesta correcta sería: -1.960917

- c) La profundidad máxima alcanzada por el cuerpo

 ✖

Una posible respuesta correcta sería: 5.121176

- d) El instante en que vuelve a la superficie

 ✖

Una posible respuesta correcta sería: 4.7

- e) El instante en el que el cuerpo se detiene

 ✖

Una posible respuesta correcta sería: 1.4

```

#2F 2022_02 T2 ( RK4 )
import numpy as np
import os
os.system('cls')
np.set_printoptions(suppress=True)
#Datos
w=8
v=0.001
y0=np.array([[0],[10]],dtype=float)#Valores iniciales y(0)=[y(0),y'(0)]
(posición y velocidad)
x0=0#tiempo inicial
p=1000
c=1.003
g=10
h=0.1
#Ec. diferencial despejada:
#d2y=0*y-c*g/w*y'+g-p*g**2*v/w
'''
No es necesario realizar modificaciones a la rutina
'''
#####RUTINA023##### Rk4
m=len(y0)
a=np.hstack((np.zeros((m-1,1)),np.eye((m-1))))
A=lambda x:np.vstack((a,np.array([0,-c*g/w]),dtype=float))
B=lambda x:np.vstack((np.zeros((m-1,1)),np.array([g-
p*g**2*v/w]),dtype=float))
x=x0
R=[]
R.append((x0,y0[0][0],y0[1][0],(A(x0)@y0+B(x0)).flatten()[1]))#Iniciali
zación matriz de cálculos
n=120
for i in range(n):
    dy0=A(x)@y0+B(x)
    x+=h/2
    y1=y0+(h/2)*dy0
    dy1=A(x)@y1+B(x)

    y2=y0+(h/2)*dy1
    dy2=A(x)@y2+B(x)

    x+=h/2
    y3=y0+h*dy2
    dy3=A(x)@y3+B(x)

    y=y0+(h/6)*(dy0+2*dy1+2*dy2+dy3)
    y0=y.copy()
    R.append((x,y0[0][0],y0[1][0],(A(x0)@y0+B(x0)).flatten()[1]))

Res=np.vstack(R)

```

```

print(Res)#Se imprime matriz de resultados para sacar respuestas
print('a-)Aceleración en el punto de máxima profundidad:',Res[14,3])
print('b-)Velocidad al llegar a la superficie',Res[47,2])
p=Res[:,1]
pmax=p[0]
tpmax=0#tiempo al que corresponde la profundidad máxima (también cuando
el cuerpo se detiene)
for i in range(len(p)):
    if pmax<p[i]:
        pmax=p[i]
        tpmax=i*h
print('c-)Profundidad máxima alcanzada por el cuerpo',pmax)
print('d-)Instante en que vuelve a la superficie:',4.7)
print('f-)El instante en que el cuerpo se detiene',tpmax)

```

```

a-)Aceleración en el punto de máxima profundidad: -2.5994531450105516
b-)Velocidad al llegar a la superficie -1.9609173341692199
c-)Profundidad máxima alcanzada por el cuerpo 5.121176835588198
d-)Instante en que vuelve a la superficie: 4.7
f-)El instante en que el cuerpo se detiene 1.4000000000000001

```

Primer final 2023 Tema 1 (Euler)

Si observamos cierta cantidad inicial de sustancia o material radioactivo, al paso del tiempo se puede verificar un cambio en la cantidad de dicho material; la cantidad M del material es una función del tiempo t , esto es $M=M(t)$.

Experimentalmente se ha llegado al conocimiento de que, en cualquier tiempo $t>0$, la rapidez de cambio de la cantidad $M(t)$ de material radioactivo es directamente proporcional a la cantidad de material presente. Si inicialmente hay 100 mg de material y, después de dos años, se observa que el 5% de la masa original se desintegró.

Utilizar el método de EULER, prepare la tabla de resultados con los elementos indicados t , M , $\frac{dM}{dt}$ con un paso de $h = 0.5[\text{años}]$, Responder el cuestionario utilizando (6) seis decimales para las respuestas sin redondear el último decimal; utilizar todos los decimales para los cálculos; no utilizar formato exponencial:

Determinar haciendo uso solo de la tabla de la resolución numérica:

- a) El tiempo necesario para que se desintegre el 10% de la masa original (2P)

x

Una posible respuesta correcta sería: 4

- b) ¿Durante cuánto tiempo el material tendrá 50% o más de su masa original?. (2P)

x

Una posible respuesta correcta sería: 27

- c) ¿Cuál es el valor de la constante de proporcionalidad k ? Halle de forma gráfica y cargue con 4 decimales. (4P)

x

Una posible respuesta correcta sería: -0.0255

- d) ¿Qué porcentaje del material radiactivo desaparecerá en 100 años?. (2P)

%

Una posible respuesta correcta sería: 92.318810

- e) Si la masa inicial del mismo material es de 200 mg, ¿durante cuánto tiempo el material tendrá 50% o más de su masa original? (2P)

x

Una posible respuesta correcta sería: 27

```

#1F 2023 T1 ( Euler )
import numpy as np
import matplotlib.pyplot as plt
import os
os.system('cls')
np.set_printoptions(suppress=True)
#Datos
h=0.5
y0=np.array([[100]],dtype=float)
M=100#Masa inicial guardada para calculos en ítems
x0=0
#Ec. Diferencial: y'=k*y donde y: cantidad de materia presente
def euler(x0,y0,k,n):
    #####RUTINA020#####
    m=len(y0)
    a=np.hstack((np.zeros((m-1,1)),np.eye((m-1))))
    A=lambda x:np.vstack((a,np.array([k]),dtype=float)))#k es una
    incógnita por eso de declara en el argumento de la función
    B=lambda x:np.vstack((np.zeros((m-1,1)),np.array([0]),dtype=float)))
    x=x0
    y=y0
    R=[]
    R.append([0,x,y[0,0],(A(x)@y0+B(x))[0,0],y[0,0]/M*100])#R=[tiempo,mate
    ria, variación de material, porcentaje de material presente]
    for i in range(n):
        dy=A(x)@y0+B(x)
        y=y0+h*dy
        y0=y.copy()
        x+=h
        R.append([i+1,x,y[0,0],(A(x)@y0+B(x))[0,0], y[0,0]/M*100 ])
    return np.vstack(R)
#la incógnita va a estar en la matriz de resultados se tiene que
invocar al elemento que le contiene a la "y"

ec=lambda k:euler(x0,y0,k,4)[4,2]#4 pasos para llegar al 2do año y
condicionar la cantidad de materia presente
#y así obtener el valor de "k"
#el ejercicio condiciona hallar el valor de la constante k por medio
del gráfico
xvals=np.linspace(-1,1,50)
yvals=[(ec(j)-95) for j in xvals]
plt.plot(xvals,yvals,label="f(k)")
plt.legend()
plt.grid(True)
plt.show()
cte=-0.0255
#Se resuelve la ecuación diferencial completamente al conseguir el
valor de "k"
Res=euler(x0,y0,cte,250)

```

```
print(Res)
print('a-)Tiempo necesario para que se desintegre el 10%:',4)
print('b-)Tiempo que el material es mayor o igual al 50% del
inicial:',27)#Se observa de la tabla para m0=100
print('c-)Valor de la cte de proporcionalidad k:',cte)
print('d-)Porcentaje del material que desaparecerá en 100 años:',100-
Res[200,4])
print('e-)Tiempo con porcentaje de material mayor o igual al 50% con
200[mg] de masa inicial:',27)#se observa de ka tabla para m0=200mg
```

```
a-)Tiempo necesario para que se desintegre el 10%: 4
b-)Tiempo que el material es mayor o igual al 50% del inicial: 27
c-)Valor de la cte de proporcionalidad k: -0.0255
d-)Porcentaje del material que desaparecerá en 100 años: 92.31881046210512
e-)Tiempo con porcentaje de material mayor o igual al 50% con 200[mg] de masa inicial: 27
```

Primer final 2023 Tema 2 (Int. Newton-Falsa Posición-Trapecio)

Consideraciones a tener cuenta para la resolución de los temas:

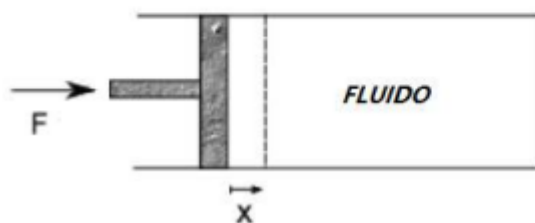
□ utilizar seis decimales para las respuestas sin redondear el ultimo decimal

□ utilizar todos los decimales para los cálculos

□ no utilizar formato exponencial

En la tabla se tienen los valores de presión y volumen de un fluido según el sistema cilindro-pistón que se muestra en la figura. Las sección y longitud del cilindro son $A=2$ y $L=4$ respectivamente, y se supone que el fluido ocupa todo el espacio disponible.

V	6	5.6	5.2	5	4.8	4	3.6	3.1	2.7	2.5
P	4.6	5	5.8	6.1	7	7.8	8	8.6	9	9.2



Utilizando los métodos de interpolación de Newton, Falsa Posición con $\text{tol}=1\text{e-}8$ y Trapecio con $s=9$ responda el cuestionario (utilizar seis decimales para las respuestas sin redondear el ultimo decimal; utilizar todos los decimales para los cálculos; no utilizar formato exponencial; No es necesario realizar ninguna conversión de unidades):

1) La presión del fluido en la posición $x=1.53$ del pistón:

6.323326



Una posible respuesta correcta sería: 6.323327

2) El trabajo realizado en valor absoluto de la posición $x=2.2$ a $x=2.35$ del pistón es:

2.510990



Una posible respuesta correcta sería: 2.51099

3) Si el pistón se detiene en $x=1.64$, la fuerza F que se debe ser aplicada para mantener el equilibrio es:

14.828572



Una posible respuesta correcta sería: 14.828572

4) La posición del pistón para una presión de $P=6.93$ es

x

Una posible respuesta correcta sería: 2.825

```
#1F 2023 T2 (Int_Newt, f_posición y trapecio)
import numpy as np
import matplotlib.pyplot as plt
import os
os.system('cls')
np.set_printoptions(suppress=True)
#Datos
V=np.array([6,5.6,5.2,5,4.8,4,3.6,3.1,2.7,2.5],dtype=float)#Volumen
Pr=np.array([4.6,5,5.8,6.1,7,7.8,8,8.6,9,9.2],dtype=float)#Presion
S=2#Sección del cilindro
L=4#Longitud del cilindro
s=9#Cantidad de trapecios
tol=1e-8#Tolerancia para newton raphson

#Cálculo de la función presión en función del volumen P(V)

#####RUTINA013#####Interpolación de
Newton
x=V
y=Pr
n=len(x)
p=np.poly1d([1,-x[0]])
P=np.poly1d([y[0]])
for i in range(1,n):
    a=(y[i]-P(x[i]))/p(x[i])
    P+=a*p
    p=np.polymul(p,np.poly1d([1,-x[i]]))

#Se puede definir la presión del fluido a partir de la posición usando
la formula del volumen del cilindro
#en función de la posición del pistón---->V=S*(L-x)
Px=lambda x:P(S*(L-x))#Se define la función Presión en función de la
posición del pistón

print('1-)La presión del fluido en la posición x=1.53 del
pistón:',Px(1.53))

#Para el trabajo realizado se tiene que la fuerza ejercida en el pistó
es  $F=P_x*S$  donde  $P_x$  es la presión y  $S$  es el área
#así  $dW=F*dx=S*P_x(x)*dx$ -----> $S*P_x(x)$  es la función a integrar por
Trapecios
```

```
#####RUTINA014#####
n=s
a,b=(2.2,2.35)
f=lambda x:S*Px(x)
h=(b-a)/n
A=0
y=f(np.linspace(a,b,n+1))
k=0
while(k<n):
    A+=(h/2)*(y[k]+y[k+1])
    k+=1

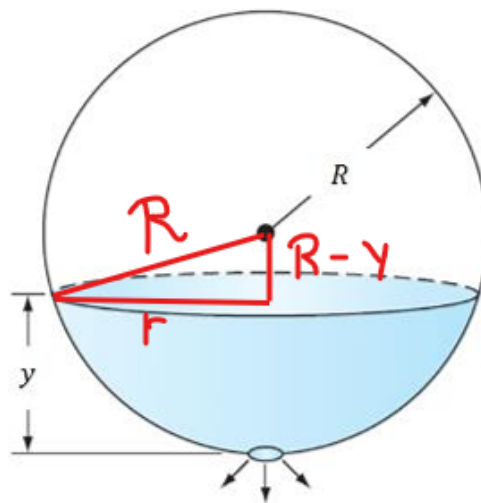
print('2-)Trabajo realizado en el intervalo [2.2,2.35]:',A)
print('3-)Fuerza sobre el pistón para mantener el
equilibrio:',S*Px(1.64))

#####RUTINA005##### Falsa posición
n=400
f=lambda x:Px(x)-6.93
#grafico para hallar intervalo de solución
xvals=np.linspace(0,3,50)
plt.plot(xvals,f(xvals))
plt.grid()
plt.show()
a,b=(2,3)
c0=a
for i in range(n):
    c=(b*f(a)-a*f(b))/(f(a)-f(b))
    err=np.abs(c0-c)
    relerr=np.abs(err/c)
    if tol>err or tol>relerr or tol>np.abs(f(c)):
        break
    else:
        if f(a)*f(c)<0:
            b=c
        else:
            a=c
    c0=c
print('4-)Posición del pistón cuando Presión=6.93:',L-c/S)
```

- 1-)La presión del fluido en la posición $x=1.53$ del pistón: 6.323326677607838
- 2-)Trabajo realizado en el intervalo $[2.2,2.35]$: 2.5109904169323256
- 3-)Fuerza sobre el pistón para mantener el equilibrio: 14.828571613703389
- 4-)Posición del pistón cuando Presión=6.93: 2.6019155008387376

Segundo final 2023 Tema 1 (RKO4 y Trapecio)

Si se drena el agua desde un tanque esférico por medio de una válvula en la base, el líquido fluirá rápido cuando el tanque esté lleno y despacio conforme se drene. Como se ve, la tasa o velocidad a la que el nivel del agua disminuye es proporcional a la raíz cuadrada de la altura del fluido dentro del tanque. Teniendo en cuenta que la constante de proporcionalidad es de $k=0.06$, la profundidad del agua y se mide en m : metros, y el tiempo t : en minutos.



Sabiendo que el nivel del fluido inicialmente es de $3[m]$, y que el paso a ser utilizado es de $h=0.5[min]$ El radio de la esfera es de $R=2[m]$.

Emplear el método de RKO4 para obtener la función tabulada de $y=f(t)$. Responder el cuestionario utilizando seis decimales para las respuestas sin redondear el último decimal; utilizar todos los decimales para los cálculos; no utilizar formato exponencial (cargar los resultados con su signo):Determinar:

1) Con la ayuda de la función $y=f(t)$, calcular el valor numérico esperado para:

1.1 El instante inmediatamente anterior al que el tanque se vacía (1P):

Respuesta para parte 1

Una posible respuesta correcta sería: 57

1.2 En $t= 1.5$ min, la altura del nivel de liquido (1P):

Respuesta para parte 2

Una posible respuesta correcta sería: 2.846140

2) Recordando que el caudal tiene como expresión a $Q=Av$ donde Q :Caudal, A : sección de la superficie libre del fluido, y v : es la velocidad del fluido a una altura y . Calcular la función tabulada de $Q=Q(t)$, calcular el valor numérico esperado para:

2.1 Determine el caudal en $t=1$ min (1P) :

Respuesta para parte 3

Una posible respuesta correcta sería: 1.025187

2.2 Determine el caudal en $t= 2$ min (1P):

Respuesta para parte 4

Una posible respuesta correcta sería: 1.061120

3) Teniendo en cuenta que el caudal es $Q=\Delta V/\Delta t$, utilizar el método del trapecio para estimar la variación de volumen en función del tiempo, calcular el valor numérico esperado para:

3.1 La variación de volumen en desde el inicio hasta $t= 0.5$ min (1P):

Respuesta para parte 5

Una posible respuesta correcta sería: 0.495761

3.2 La variación de volumen desde $t= 1.5$ min hasta $t= 3$ min (1P):

Respuesta para parte 6

Una posible respuesta correcta sería: 1.601465

3.3 La cantidad total de volumen desalojado del tanque (1P):

Respuesta para parte 7

Una posible respuesta correcta sería: 28.273273

4) Utilizando las propiedades geométricas, calcular el valor numérico esperado para:

4.1 El radio de la superficie libre del fluido en $t= 0.5$ min (1P) :

Respuesta para parte 8

Una posible respuesta correcta sería: 1.760907

4.2 El radio de la superficie libre del fluido en $t= 1.5$ min (1P):

Respuesta para parte 9

Una posible respuesta correcta sería: 1.812193

5) Determinar la velocidad del fluido, calcular el valor numérico esperado para:

5.1 En $t=4.5$ min (1P):

Respuesta para parte 10

Una posible respuesta correcta sería: -0.095823

5.2 En $t=10$ min (1P):

Respuesta para parte 11

Una posible respuesta correcta sería: -0.085923

6) La velocidad máxima con la que disminuye la altura del fluido es de: (1P):

Respuesta para parte 12

Una posible respuesta correcta sería: -0.103923

```
#2F 2023 T1 ( RK4 y Trapecio )
import numpy as np
import os
os.system('cls')
np.set_printoptions(suppress=True)
#Datos
k=0.06
y0=np.array([3],dtype=float)#t0=[altura del líquido]
h=0.5
r=2
x0=0
#Ec. diferencial: y'=-k*(y)**0.5

#####RUTINA023#####RK4
m=len(y0)
a=np.hstack((np.zeros((m-1,1)),np.eye((m-1))))
A=lambda x:np.vstack((a,np.array([0]),dtype=float)))
B=lambda x:np.vstack((np.zeros((m-1,1)),np.array([-k*(x)**0.5]),dtype=float)))
x=x0
R=[]
R.append([x,y0[0],(A(x)@y0+B(y0))[0,0]])
n=114#al poner un valor mayor a esto ya devuelve "nan" por eso utilizo
el límite directo con ese valor
for i in range(n):
    dy0=A(x)@y0+B(y0[0])
    x+=h/2
    y1=y0+(h/2)*dy0
    dy1=A(x)@y1+B(y1[0])
    y2=y0+(h/2)*dy1
    dy2=A(x)@y2+B(y2[0])
    x+=h/2
    y3=y0+h*dy2
    dy3=A(x)@y3+B(y3[0])
    y=y0+(h/6)*(dy0+2*dy1+2*dy2+dy3)
    y0=y.copy()
```

```

R.append([x,y0[0,0],(A(x)*y0+B(y0[0]))[0,0]])
Res=np.vstack(R)
print(Res)
print('1.1-)Instante inmediato inferior al que el tanque se vacía:',57)
print('1.2-)Nivel del líquido en t=1.5[min]:',Res[3,1])
#Cálculo de la tabla de caudal en función del tiempo: Q=Area.(y')
Q=Res[:,2]*(r**2-(r-Res[:,1])**2)*np.pi
print('2.1-)Caudal en t=1[min]:',Q[2])
print('2.2-)Caudal en t=2[min]:',Q[4])
def trapecio(y,h):
    #####RUTINA014##### Método del
    trapecio
    k=0
    n=len(y)-1
    A=0
    while(k<n):
        A+=(h/2)*(y[k]+y[k+1])
        k+=1
    return A
print('3.1-)Variación de volumen en [0,0.5][s]:',trapecio(Q[0:2],0.5))
print('3.2-)Variación de volumen en [1.5,3][s]:',trapecio(Q[3:7],0.5))
print('3.3-)Volumen total desalojado:',trapecio(Q,0.5))
#Radio de la superficie libre de líquido
Rad=(r**2-(r-Res[:,1])**2)**0.5
print('4.1-)Radio de la superficie libre de líquido en
t=0.5[min]:',Rad[1])
print('4.2-)Radio de la superficie libre de líquido en
t=1.5[min]:',Rad[3])
print('5.1-)Velocidad del fluido en 5=4.5[min]:',Res[9,2])
print('5.2-)Velocidad del fluido en 5=10[min]:',Res[20,2])
print('6-)Velocidad máxima con la que disminuye el
fluido:',np.min(Res[:,114,2]))

```

```

1.1-)Instante inmediato inferior al que el tanque se vacía: 57
1.2-)Nivel del líquido en t=1.5[min]: 2.8461404273424353
2.1-)Caudal en t=1[min]: -1.025187424925576
2.2-)Caudal en t=2[min]: -1.061120684347358
3.1-)Variación de volumen en [0,0.5][s]: -0.4957611030285748
3.2-)Variación de volumen en [1.5,3][s]: -1.6014659582518622
3.3-)Volumen total desalojado: -28.273271638172144
4.1-)Radio de la superficie libre de líquido en t=0.5[min]: 1.760907828507886
4.2-)Radio de la superficie libre de líquido en t=1.5[min]: 1.812193802333724
5.1-)Velocidad del fluido en 5=4.5[min]: -0.09582304845554265
5.2-)Velocidad del fluido en 5=10[min]: -0.08592304845807021
6-)Velocidad máxima con la que disminuye el fluido: -0.10392304845413262

```

No se pudo resolver porque se necesita el valor inicial de la velocidad pero el enunciado no lo da.

Consideraciones a tener cuenta para la resolución de los temas:

- * utilizar seis decimales para las respuestas redondeando el ultimo decimal
- * utilizar todos los decimales para los cálculos
- * no utilizar formato exponencial
- * utilizar una tolerancia de 10^{-8} para los cálculos
- * no es necesario realizar ninguna conversión de unidades

UTILIZAR EL METODO DE EULER $Z_{k+1} = Z_k + h \frac{dZ}{dt}$

Suponga que un proyectil se lanza hacia arriba desde la superficie de la tierra. Se acepta que la única fuerza que actúa sobre el objeto es la fuerza de la gravedad, hacia abajo. En estas condiciones, se usa un balance de fuerza para obtener,

$$\frac{dv}{dt} = -\frac{gR^2}{(R+x)^2}$$

donde v = velocidad hacia arriba (m/s), t = tiempo (s), x = altitud (m) medida hacia arriba a partir de la superficie

Determinar:

Con respecto a la resolución de la EDO tomando como $x = x(t)$; $v = v(t)$ con el método de EULER con un paso de $h = 0.5$ segundos. Los valores obtenidos guardar en una matriz de respuestas de la forma $\text{Resp}=[t \ x \ v \ dv/dt]$, responder:

1. Cuál es la posición a los 32 segundo (2P)

m

×

Una posible respuesta correcta sería: 37637.312625

2. Cuál es la velocidad a los 42 segundos (2P)

m/s

×

Una posible respuesta correcta sería: 921.292443

3. Cuál es la aceleración del proyectil a los 47 segundos (2P)

m/s²

×

Una posible respuesta correcta sería: -9.649258

Con ayuda de la matriz de respuesta (tres valores antes y tres valores después de que llega a la altura máxima) construir los valores iniciales de

$T=[t_1 \ t_2 \ t_3 \ t_4 \ t_5 \ t_6]$; $X=[x_1 \ x_2 \ x_3 \ x_4 \ x_5 \ x_6]$ y $V=[v_1 \ v_2 \ v_3 \ v_4 \ v_5 \ v_6]$

(recuerde que v_1, v_2 y v_3 son positivos; v_4, v_5 y v_6 son negativos, todos son valores consecutivos de la matriz de *Resp* de la sección anterior)

En base a los vectores anteriores y utilizando el método de interpolación de Newton para generar las funciones de $x = x(t)$; $v = v(t)$

Y para determinar las raíces utilizar el método de la SECANTE con una tolerancia de 10^{-8} y valores iniciales (valores del tiempo donde la velocidad cambia de signo) del

$p_0=t_3$ y $p_1=t_4$

responder:

4. La altura máxima alcanzada por el cuerpo (2P)

m

×

Una posible respuesta correcta sería: 91772.87563

5. El instante en que el proyectil llega a la altura máxima (2P)

6. La aceleración del proyectil al llegar a la altura máxima (2P)

m/s²

×

Una posible respuesta correcta sería: -9.530412

3er final 2023 (Rk4 , Hamming modificado y simpson 3/8).

Una cuerpo que pesa 4 [N] está unida a un resorte que cuelga del techo y cuya constante es 2 [N/m]. El medio ofrece una fuerza de amortiguamiento que es numéricamente igual a la velocidad instantánea. La masa inicialmente se libera desde un punto localizado a 1 [m] por encima de la posición de equilibrio estático a velocidad descendente de 8 [m/s].

Consideraciones: $g = 10[m/s^2]$

El sistema de referencia, considerar el origen en el punto donde el resorte esta sin deformación, con el eje oy en dirección vertical y con sentido positivo hacia arriba. Utilizar un paso de $h=0.1$ [s]. RK04 para los valores iniciales, y el método de Hamming Modificado.

Responder el cuestionario utilizando seis decimales para las respuestas sin redondear el último decimal; utilizar todos los decimales para los cálculos; no utilizar formato exponencial (cargar los resultados con su signo):

Calcular:

1) el tiempo justo antes de que la masa cruza por la primera posición de equilibrio. (2P).

1.1

2) Encuentre el momento en el cual la masa logra su desplazamiento extremo por primera vez. (2P)

0.6

3) ¿Cuál es la posición de la masa en ese instante? Cargar el valor con su signo. (1P)

3.333103

4) Utilizando la tabla de velocidad funcion del tiempo, en el movimiento, determine el desplazamiento realizado por el cuerpo, en base la función tabulada de $dy/dt=f(t)$. Aplicar el método de Simpson 3/8. Utilizar el paso de 0.1 [s].

4.1 - 4) Desplazamiento efectuado desde $t_1=0.8$ a $t_2=1.7$ (con ayuda del método de integración). Cargar el valor con su signo. (2P)

1.186821

4.2 - 5) Desplazamiento real del cuerpo desde $t_1=0.8$ a $t_2=1.7$.

$\Delta y = y_f - y_0$. Cargar el valor con su signo. (2P)

1.186827

4.3 - 6) Error cometido entre ambos métodos de los ítem 4.1 y 4.2. Cargar el valor absoluto. (1P)

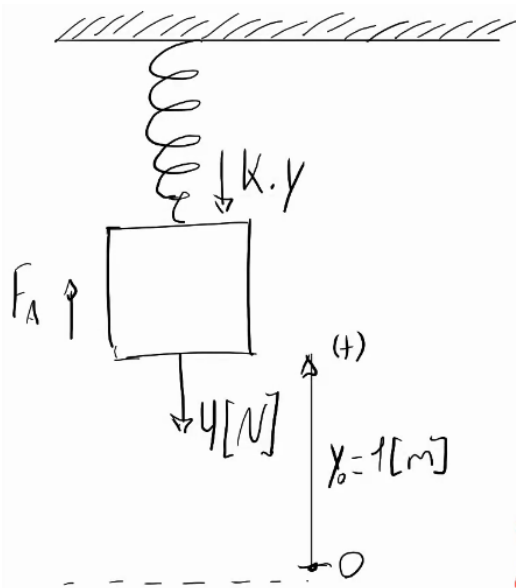
0.000005

7. Determine la fuerza resultante sobre el cuerpo en $t=2.0$ [s]. Cargar el valor con su signo. (2P)

-0.617377

8) Determinar el instante, justo antes que llegue a la segunda posición de equilibrio dinámico. (2P)

2.8



$$k = 2 \text{ N/m}$$

F_A : Fuerza de amortiguamiento

$$\sum F = m \cdot a$$

$$-4 - y' - k \cdot y = \frac{4}{g} \cdot y''$$

Amortiguación siempre en sentido contrario a la velocidad.

```
#3F 2023 ( RK4 y Trapecio)
import numpy as np
import os
os.system('cls')
np.set_printoptions(suppress=True)

#Datos
p=4
k=2
h=0.1
g=10
y0=np.array([[1],[-8]],dtype=float)#y0=[posición, velocidad]
x0=0
#Ec. diferencial: y''=-k*g*y/4-g*y'/4-g

#Rk4 para puntos necesarios en Hamming M0idificado
#####RUTINA023#####
m=len(y0)
a=np.hstack((np.zeros((m-1,1)),np.eye((m-1))))
A=lambda x:np.vstack((a,np.array((-k*g/4,-g/4)),dtype=float))
B=lambda x:np.vstack((np.zeros((m-1,1)),np.array((-g)),dtype=float))
x=x0
yn=[]
yn.append(y0)
R=[]
R.append([x,y0[0,0],y0[1,0],(A(x)@y0+B(x))[1,0]])
n=3
for i in range(n):
    dy0=A(x)@y0+B(x)
    x+=h/2
    y1=y0+(h/2)*dy0
    dy1=A(x)@y1+B(x)
```

```

y2=y0+(h/2)*dy1
dy2=A(x)@y2+B(x)
x+=h/2
y3=y0+h*dy2
dy3=A(x)@y3+B(x)
y=y0+(h/6)*(dy0+2*dy1+2*dy2+dy3)
y0=y.copy()
R.append([x,y0[0,0],y0[1,0],(A(x)@y0+B(x))[1,0]])
yn.append(y0)
#####RUTINA027#####
y0,y1,y2,y3=yn
x1=x0+h
x2=x1+h
x3=x2+h
n=47
for i in range(n):
    dy1=A(x1)@y1+B(x1)
    dy2=A(x2)@y2+B(x2)
    dy3=A(x3)@y3+B(x3)
    p=y0+(4/3)*h*(2*dy1-dy2+2*dy3)
    if i>0:
        mod=p+(112/121)*(y3-p0)
    else:
        mod=p
    x4=x3+h
    dy4=A(x4)@mod+B(x4)
    y=(9*y3-y1)/8+(3/8)*h*(-dy2+2*dy3+dy4)
    y0=y1.copy()
    y1=y2.copy()
    y2=y3.copy()
    y3=y.copy()
    x1=x2
    x2=x3
    x3=x4
    p0=p.copy()
    R.append([x4,y[0,0],y[1,0],(A(x4)@y+B(x4))[1,0]])
Res=np.vstack(R)
print(Res)
print('1-)Tiempo antes de que la masa cruce por la primera posición de
equilibrio:',1.5)#Equilibrio: aceleración=0
print('2-)Instante de desplazamiento extremo por primera vez:',1)#Está
entre 1.1[s] y 1[s] yo elegí el tiempo antes
print('3-)Posición de la masa en ese instante:',Res[10,1])
def simp(y,h):#simpson 3/8
#####RUTINA016#####
A=0
k=0
n=len(y)-1
while(k<n):

```

```

    A+=(3*h/8)*(y[k]+3*y[k+1]+3*y[k+2]+y[k+3])
    k+=3
    return A

```

```

d1=simp(Res[8:18,2],0.1)
print('4.1-4)Desplazamiento en intervalo [0.8,1.7][s]',d1)
d2=Res[17,1]-Res[8,1]
print('4.2-5)Desplazamiento real en intervalo [0.8,1.7][s]',d2)
print('4.3-6)Error cometido:',np.abs(d2-d1))
print('7-)Fuerza resultante sobre el cuerpo:',Res[20,3]*4/g)
print('8-)Instante justo antes de llegar a la segunda posición de
equilibrio dinámico:',3.2)

```

- 1-)Tiempo antes de que la masa cruce por la primera posición de equilibrio: 1.5
- 2-)Instante de desplazamiento extremo por primera vez: 1
- 3-)Posición de la masa en ese instante: -2.8706353921837366
- 4.1-4)Desplazamiento en intervalo [0.8,1.7][s] 0.38779868182918303
- 4.2-5)Desplazamiento real en intervalo [0.8,1.7][s] 0.38788933453661345
- 4.3-6)Error cometido: 9.065270743041642e-05
- 7-)Fuerza resultante sobre el cuerpo: -0.4590490235871044
- 8-)Instante justo antes de llegar a la segunda posición de equilibrio dinámico: 3.2

Segundo final 2024-01 Tema2 (Hamming+RK04)

Sea $P(t)$ el número de individuos en una población en el tiempo t , medido en años. Si el índice de natalidad b es $b = 0.039 \cdot (2 + \sin(\frac{\pi t}{3}))$ y el índice de mortalidad d es proporcional al tamaño de la población (debido a la superpoblación), entonces el índice de crecimiento de la población está dado por la ecuación logística

$$\frac{dP(t)}{dt} = b \cdot P(t) - k[P(t)]^2, \text{ con } 0 \leq t \leq 6 \text{ años}$$

Donde $P(0) = 50976$, y $k = 1.4 \times 10^{-7}$.

Utilizando el método de Hamming con un paso de $h = 1$ año y de Interpolación de Newton, responda el cuestionario: (se debe utilizar RK4 para hallar los valores faltantes)

(para las respuestas utilice seis decimales redondeando el último dígito; use todos los decimales para los cálculos; no utilice formato exponencial)

Responda las siguientes preguntas.

1. La población (cantidad de personas) a los 2 años

✓

Una posible respuesta correcta sería: 62026.392082

2. La población (cantidad de personas) a los 6 años

✓

Una posible respuesta correcta sería: 76966.806106

#3F 2023 (Hamming y RK4)

```
import numpy as np
```

```
import os
```

```
os.system('cls')
```

```
np.set_printoptions(suppress=True)
```

```
#Datos
```

```
y0=np.array([50976],dtype=float)#y0=[población]
```

```
k=1.4*1e-7
```

```
h=1
```

```
x0=0
```

```
#Ec. diferencial:p'=0.039*( 2+np.sin(np.pi*t/3) )*p-k*p**2
```

```
#Rk4 para puntos necesarios en Hamming
```

```
#####RUTINA023#####
```

```
m=len(y0)
```

```
a=np.hstack((np.zeros((m-1,1)),np.eye((m-1))))
```

```
A=lambda x:np.vstack((a,np.array(([0]),dtype=float)))
```

```
B=lambda x,p:np.vstack((np.zeros((m-1,1)),np.array([(0.039*(  
2+np.sin(np.pi*x/3) )*p-k*p**2]),dtype=float)))
```

```
x=x0
```

```
yn=[]
```

```
yn.append(y0)
```

```
R=[]
```

```
R.append([x,y0[0],(A(x)@y0+B(x,y0[0]))[0,0]])
```

```
n=3
```

```
for i in range(n):
```

```
    dy0=A(x)@y0+B(x,y0[0])
```

```
    x+=h/2
```

```
    y1=y0+(h/2)*dy0
```

```
    dy1=A(x)@y1+B(x,y1[0])
```

```
    y2=y0+(h/2)*dy1
```

```

dy2=A(x)@y2+B(x,y2[0])
x+=h/2
y3=y0+h*dy2
dy3=A(x)@y3+B(x,y3[0])
y=y0+(h/6)*(dy0+2*dy1+2*dy2+dy3)
y0=y.copy()
yn.append(y0)
R.append([x,y0[0,0],(A(x)@y0+B(x,y[0]))[0,0]])

#####ROUTINA027##### Hamming
y0,y1,y2,y3=yn
x1=x0+h
x2=x1+h
x3=x2+h
n=3
for i in range(n):
    dy1=A(x1)@y1+B(x1,y1[0])
    dy2=A(x2)@y2+B(x2,y2[0])
    dy3=A(x3)@y3+B(x3,y3[0])
    p=y0+(4/3)*h*(2*dy1-dy2+2*dy3)
    x4=x3+h
    dy4=A(x4)@p+B(x4,p[0])# mod se debe evaluar en B en este caso
    y=(9*y3-y1)/8+(3/8)*h*(-dy2+2*dy3+dy4)
    y0=y1.copy()
    y1=y2.copy()
    y2=y3.copy()
    y3=y.copy()
    x1=x2
    x2=x3
    x3=x4
    p0=p.copy()
    R.append([x4,y[0,0],(A(x4)@y+B(x4,y[0]))[0,0]])
Res=np.vstack(R)
print(Res)
print('1-)Población a los 2 años:',Res[2,1])
print('2-)Población a los 6 años:',Res[6,1])

```

1-)Población a los 2 años: 62026.39208216393

2-)Población a los 6 años: 76966.8061058251

Tercer final 2024-01 Tema1 (Bisección y simpson1/3)

Pregunta 1

Parcialmente
correcta

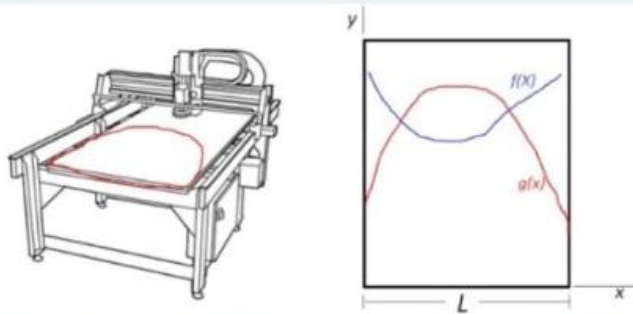
Se puntúa 7,50
sobre 10,00

🚩 Marcar
pregunta

Consideraciones a tener cuenta para la resolución de los temas:

- ☐ **utilizar seis decimales para las respuestas y redondear el ultimo decimal**
- ☐ **utilizar todos los decimales para los cálculos**
- ☐ **no utilizar formato exponencial**
- ☐ **no es necesario realizar ninguna conversión de unidades**
- ☐ **utilizar solo y exclusivamente los métodos solicitados**

La maquina CNC mostrada en la figura se utiliza para realizar cortes de chapa. Debido a una avería la trayectoria de la maquina esta limitada por la ecuación $f = (x - 2)^2 + e^{x-5} + 5$, y solo la chapa se puede desplazar cuando sea necesario.



Sabiendo que la forma de la chapa esta descrita por la ecuación $g(x) = -(x - 3)^2 + 10 + \frac{1}{x+1}$ y $L = 4.7$ m, responda el cuestionario utilizando el método de la Bisección con una tolerancia de 10^{-6} y de Simpson 1/3 con $s = 12$:

1- En las condiciones iniciales, la chapa disponible para el corte es:

38.102845 ✓

Una posible respuesta correcta sería: 38.102845

2- En las condiciones iniciales, el corte de chapa de mayor dimensión es:

28.513984 ✓

Una posible respuesta correcta sería: 28.513978

3- La menor distancia que la chapa se debe desplazar hacia abajo tal que este fuera de la trayectoria de corte es:

5.201810 ✓

Una posible respuesta correcta sería: 5.20181

4- Independientemente de la chapa, la mayor área de corte que se puede obtener de la maquina es:

71.564598 ✗

Una posible respuesta correcta sería: 33.461753

```

#3F 2024_01 T1 ( Bisección y simpson 1/3)
import numpy as np
import matplotlib.pyplot as plt
from jax import grad
import jax.numpy as jnp
import os
os.system('cls')
np.set_printoptions(suppress=True)
#Datos
L=4.7
s=12#Polinomios para simpson 1/3
tol=1e-6
def f(x):
    return (x-2)**2+jnp.exp(x-5)+5
def g(x):
    return -(x-3)**2+10+1/(x+1)
def bis(f,a,b,n):
    #####RUTINA004##### Bisección
    c0=a
    for i in range(n):
        c=(a+b)/2
        err=np.abs(c0-c)
        relerr=np.abs(err/c)
        if tol>err or tol>relerr or tol>np.abs(f(c)):
            break
        else:
            if f(a)*f(c)<0:
                b=c
            else:
                a=c
        c0=c
    return c

def simp(f,a,b,n):
    #####RUTINA016#####
    h=(b-a)/n
    A=0
    y=f(np.linspace(a,b,n+1))
    k=0
    while(k<n):
        A+=(h/3)*(y[k]+4*y[k+1]+y[k+2])
        k+=2
    return A

#gráfico para sacar intervalo de intersección entre funciones
xvals=np.linspace(0,4.7,30)
plt.plot(xvals,f(xvals),label="f(x)")
plt.plot(xvals,g(xvals),label="g(x)")
plt.legend()

```

```

plt.grid()
#plt.show()
c1=bis(lambda x:f(x)-g(x),0,1,100)
c2=bis(lambda x:f(x)-g(x),3,4,100)
#Area disponible para corte
print('1-)Chapa disponible para corte:',simp(g,0,L,2*s))
#para ítem 2-) es suma de areas para sacar el pedazo mas grande de
chapa de g(x)
print('2-)Corte de chapa de mayor dimesión
es:',simp(g,0,c1,2*s)+simp(f,c1,c2,2*s)+simp(g,c2,L,2*s))
#para ítem 3 se calcula el máximo valor de f y g, la distancia entre
ellos es la mínima a desplazar para que ya no haya corte
Xfmax=bis(grad(f),0.0,L,200)
Xgmax=bis(grad(g),0.0,L,200)
print('3-)La menor distancia que la chapa se debe desplazar:',g(Xgmax)-
f(Xfmax))
print('4-)Mayor area de corte de la máquina:',simp(f,0,L,2*s))

```

1-)Chapa disponible para corte: 38.102844613917426

2-)Corte de chapa de mayor dimesión es: 28.513971

3-)La menor distancia que la chapa se debe desplazar: 5.2018094

4-)Mayor area de corte de la máquina: 33.461754

Tercer final 2024-01 Tema 2 (Simpson 1/3 y falsa posición)

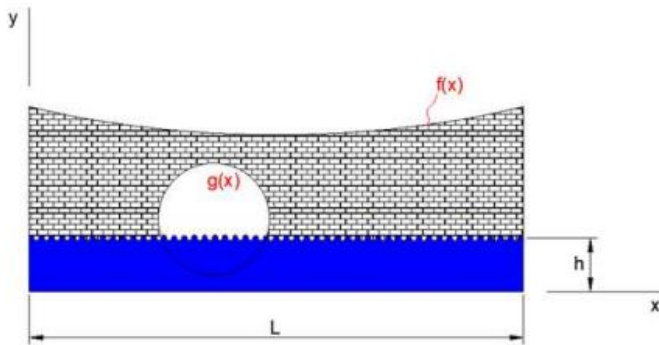
Consideraciones a tener cuenta para la resolución de los temas:

- ☐ utilizar seis decimales para las respuestas sin redondear el ultimo decimal
- ☐ utilizar todos los decimales para los cálculos
- ☐ no utilizar formato exponencial
- ☐ no es necesario realizar ninguna conversión de unidades

En la figura se puede observar la vista seccional de un desagüe cloacal de longitud $L = 12$ m con un nivel de agua a una altura $h = 4.2$ m que se desplaza a una velocidad de $v = 1.32$ m/s. La forma curva de la pared del desagüe y el orificio por donde circula el agua están descritas por las ecuaciones:

(Orificio $g(x)$) $x^2 - 12.28x + 36.34 + (y - 5.18)^2 = 0$

(Pared de desagüe $f(x)$) $y = 0.05x^2 + 8 - 0.5x + (x + 2)^{-1}$



Utilizando los métodos de Simpson 1/3 con $s=12$ y el método de la falsa posición con $\text{tol}=1\text{e-}8$, responda el cuestionario:

1. La sección libre de agua del orificio del desagüe es:

×

Una posible respuesta correcta sería: 4.096869

2. La sección de pared libre de agua es:

×

Una posible respuesta correcta sería: 36.249155

3. El caudal de agua es:

×

Una posible respuesta correcta sería: 0.210397

4. La altura máxima a la que pueda llegar el nivel de agua sin ingresar al orificio es:

×

Una posible respuesta correcta sería: 4.016036

```

#3F 2024_01 T2 (simpson tercio y falsa posición)
import numpy as np
import os
import matplotlib.pyplot as plt
from jax import grad
os.system('cls')
np.set_printoptions(suppress=True)
#funciones
def g(x,y):
    return x**2-12.28*x+36.34+(y-5.18)**2
def f(x):
    return 0.05*x**2+8-0.5*x+(x+2)**(-1)
#tolerancia
tol=1e-8
#altura del agua
h=4.2
#Velocidad de corriente de agua
v=1.32
L=12
def graficar(f,a,b,m,nombre):
    plt.plot(np.linspace(a,b,m),f(np.linspace(a,b,m)),label=nombre)
    plt.legend()
    plt.grid()
    return
def f_pos(f,a,b,n):
    #####RUTINA005#####
    c0=a
    for i in range(n):
        c=(b*f(a)-a*f(b))/(f(a)-f(b))
        err=np.abs(c0-c)
        relerr=np.abs(err/c)
        if tol>err or tol>relerr or tol>np.abs(f(c)):
            break
        else:
            if f(a)*f(c)<0:
                b=c
            else:
                a=c
        c0=c
    return c
def simp(f,a,b,n):
    #####RUTINA015##### Simpson 1/3
    A=0
    k=0
    h=(b-a)/n
    y=f(np.linspace(a,b,n+1))
    while(k<n):
        A+=(h/3)*(y[k]+4*y[k+1]+y[k+2])

```

```

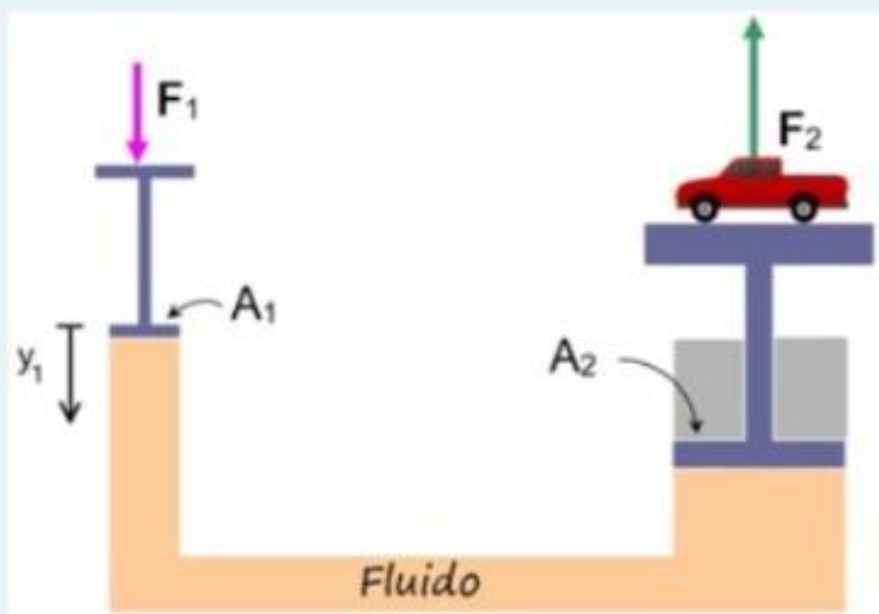
    k+=2
    return A
#cálculo de los valores de x de la cia a un nivel h
graficar(lambda x:g(x,h),0,9,50,"g(x)")
plt.show()
x1,x2=(f_pos(lambda x:g(x,h),5,6,100),f_pos(lambda x:g(x,h),6.5,7,100))
#abscisa del centro de la cia:
cx=(x1+x2)/2
#cálculo de la ordenada del centro de la cia
graficar(lambda x:g(cx,x),0,15,50,"g(y)")
plt.show()
y1,y2=(f_pos(lambda x:g(cx,x),4,5,100),f_pos(lambda x:g(cx,x),6,7,100))
#Radio de la cia:
r=(y2-y1)/2.0
#Area bajo la intersección del nivel del agua y la cia:
A1=simp(lambda x:-(-x**2+12.28*x-36.24)**0.5+5.18,x1,x2,2*12)
#area rectangular del agua en el mismo intervalo
A2=(x2-x1)*h
print('1-)Sección libre de agua:',np.pi*r**2-(A2-A1))
print('2-)Pared libre de agua:',simp(f,0,L,2*12)-L*h-(np.pi*r**2-(A2-A1)))
print('3-)Caudal del agua:',v*(A2-A1))
print('4-)Altura máxima del agua:',y1)

```

1-)Sección libre de agua: 4.055790687791311
 2-)Pared libre de agua: 36.29023404896051
 3-)Caudal del agua: 0.28448462856875856
 4-)Altura máxima del agua: 4.01398113438296

2F 2024_01 (Bisección + Simpson 3/8)

En la figura se observa un sistema hidráulico en su estado inicial. Se aplica una fuerza vertical $F_1 = \frac{y_1}{y_1^2+1} + 0.04y_1 + 100$, desplazando el pistón 1 una distancia $y_1 = -t^3 + \sqrt{t+5} + 10t^2$ en función del tiempo hacia abajo, para poder desplazar la camioneta verticalmente hacia arriba.



Considerando la masa del fluido constante e incompresible y sabiendo que $A_2 = 9.25A_1$, responda el cuestionario utilizando los métodos de la bisección con una tolerancia de 10^{-6} y el método de Simpson 3/8 con $s = 5$:

1- El desplazamiento de la camioneta cuando se detiene es:

151.5643 ✖

2- Del ítem anterior, el tiempo en el cual se da es:

6.6739 ✔

3- El módulo de F_2 cuando la camioneta se detiene es:

981.1398 ✖

4- El trabajo que realiza F_2 sobre la camioneta desde el estado inicial hasta el equilibrio es:

6199.3594 ✖

5- La rapidez máxima de ascenso de la camioneta es:

33.5065 ✖

```

#2F 2024_01 (Bisección + Simpson 3/8)
import numpy as np
import os
import matplotlib.pyplot as plt
from jax import grad
#os.system('cls')
np.set_printoptions(suppress=True)
#funciones
def F(x):
    return x/(x**2+1)+0.04*x+100
def y(x):
    return -x**3+(x+5)**(1/2)+10*x**2
#Relaciones entre areas A2=9.25*A1
#Tiempo en el cual se detiene el pistón 1 se halla mediante su
velocidad

v=grad(y)
acel=grad(v)
xvals=np.linspace(0,20,40)
v_vals=[v(i) for i in xvals]
a_vals=[acel(i) for i in xvals]
plt.plot(xvals,y(xvals),label="y(t)")
plt.plot(xvals,v_vals,label="v(t)")
plt.plot(xvals,a_vals,label="a(t)")
plt.legend()
plt.grid()
plt.show()
def bisec(f,a,b,n):
    #####RUTINA004##### Bisección
    tol=1e-6
    c0=a
    for i in range(n):
        c=(a+b)/2
        err=np.abs(c0-c)
        relerr=np.abs(err/c)
        if tol>err or tol>relerr or tol>np.abs(f(c)):
            break
        else:
            if f(a)*f(c)<0:
                b=c
            else:
                a=c
            c0=c
    return c
#intervalo:
a,b=(6.0,7.0)
#Tiempo en que se detiene la camioneta V(t)=0
tv0=bisec(v,a,b,100)

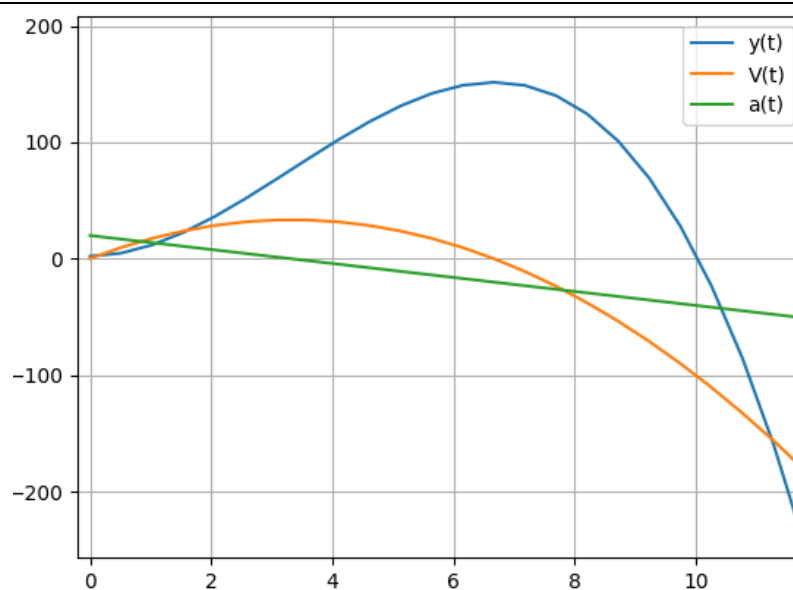
```

```

#relación entre posiciones de las superficies:  $A1*(y2(t)-y1(0))=A2*(y2(t)-y2(0))$ ---> $A1/A2$ 
d1=(y(tv0)-y(0))# variación de volumen en cada pistón debe ser iguales
 $A1*d1=A2*d2$ -----> $A1*(y(t)-y(0))=A2*d2$ 
#---> $A1*d1/A2=d2$  siendo d1 y d2 desplazamientos
print('1-)Desplazamiento de la camioneta:',d1/9.25)
print('2-)Tiempo en que ocurre',tv0)
#la fuerza en el pistón 2: igualdad de presiones  $F2/A2=F1/A1$ ----
> $F2=F1*(A2/A1)$ 
F2=lambda x:F(y(x))*9.25
print('3-)F2 cuando la camioneta se detiene:',F2(tv0))
#integración de la fuerza en el pistón 2
#simpson 3/8 (integración de la fuerza)

#####RUTINA016#####
a,b=(0,tv0)
n=3*5
h=(b-a)/n
A=0
f=lambda x:F2(x)
y=f(np.linspace(a,b,n+1))
k=0
while(k<n):
     $A+=(3*h/8)*(y[k]+3*y[k+1]+3*y[k+2]+y[k+3])$ 
    k+=3
print('4-)Trabajo que realiza F2:',A)
#Para v_máx----> $a(t)=0$  se obtiene que se encuentra en el intervalo
[3,4]
t_vmax=bisec(acel,3.0,4.0,100)
print('5-)La rapidez máxima:',v(t_vmax))

```



```

1-) 16.38533335769813
2-) 6.673976898193359
3-) 981.1398309495971
4-) 6366.54182645753
5-) 33.50655

```


Segundo final 2024-02 Tema1 (RK4)

El agua fluye desde un tanque cónico invertido con un orificio circular a una velocidad de:

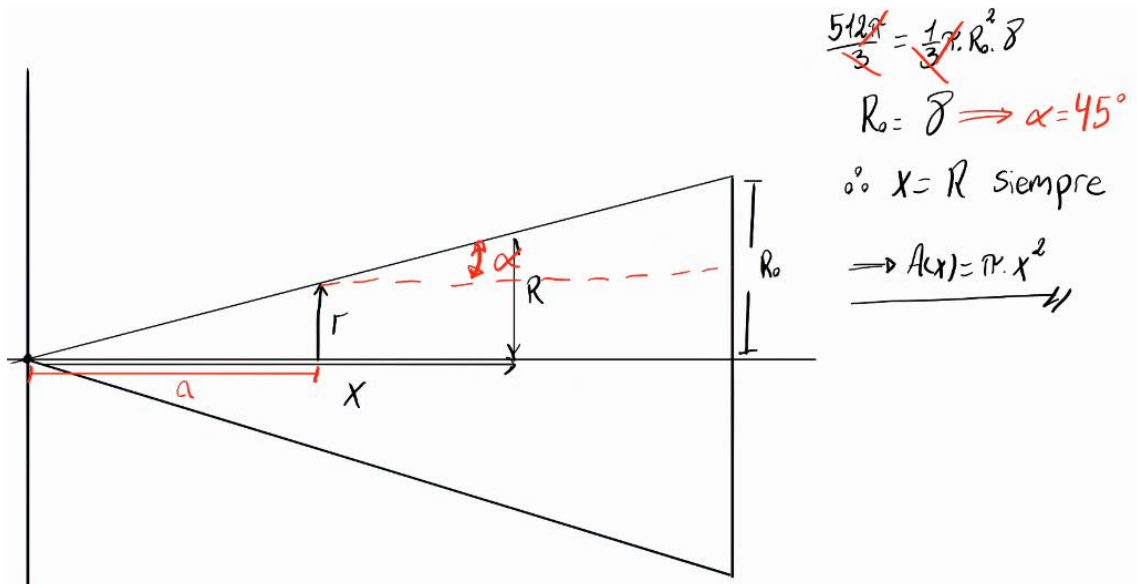
$$\frac{dx}{dt} = -0.5\pi r^2 \sqrt{2g} \frac{\sqrt{x}}{A(x)}$$

donde r es el radio del orificio, x es la altura del nivel de líquido desde el vértice del cono y $A(x)$ es el área de la sección transversal del tanque x unidades por encima del orificio. Suponga que $r = 0.1$ pies, $g = 32.1$ pies/s², y el tanque tiene un nivel inicial de agua de $\frac{512\pi}{3}$ pies y un volumen inicial de $\frac{512\pi}{3}$ pies³. Utilice el método Runge-Kutta de orden 4, y un paso de $h = 20$ segundos, para encontrar lo siguiente:

Responder las siguientes preguntas utilizando los instantes tabulados:

1. Altura del nivel del líquido en pies, a los 60 segundos :
 $x =$ [pies]
2. Volumen de líquido en pies³, a los 80 segundos:
 $Volumen =$ [pies³]

3. El área de la superficie libre del líquido en pies², a los 100 segundos
 $Area =$ [pies²]
4. El tiempo (en minutos) en que se descarga el tanque (utilice el último instante en que la altura es positiva)
 $t_{desc} =$ [min]
5. La rapidez (en in/s) con la que disminuye el nivel de la superficie del líquido, a los 20 segundos:
 $Rapidez =$ [in/s]



```

#2F 2024_02 T1 ( Rk4 )
import numpy as np
import os
os.system('cls')
np.set_printoptions(suppress=True)
#Datos
y0=np.array([8.0],dtype=float)#la posición inicial de la altura medida
desde el vértice (para situaciones de descarga
#con conos se considera despreciable la distancia entre el vértice real
del cono y el pequeño orificio que tiene para descargarse)
x0=0
g=32.1
h=20
r=0.1
#Ec diferencial
#La ec. diferencial da una función que depende del área de la sección
del cono
#a través de la geometría se halla que  $A(x)=\pi x^2$  con x siendo la
altura del nivel del agua
def dxdt(x):
    return -0.5*np.pi*(r**2)*np.sqrt(2*g*x)/(np.pi*(x)**2)

#####RUTINA023##### Rk4
m=len(y0)
a=np.hstack((np.zeros((m-1,1)),np.eye((m-1))))#Uno puede elegir seguir
declarando a la matriz "A" o simplemente eliminarlo de todas las líneas
cuando se trata de un ejercicio donde
#Toda la ecuación diferencial fué llevada en la matriz "B"
A=lambda x:np.vstack((a,np.array([0]),dtype=float))
'''Voy a utilizar "d" como la variable de altura del nivel del agua
para que "x" no se confunda con la variable del tiempo que se mueve
un paso h'''
B=lambda d:np.vstack((np.zeros((m-1,1)),np.array(dxdt(d),dtype=float)))
x=x0
n=100
R=[]
R.append([x,y0[0],B(y0[0])[0][0]])#Tener en cuenta que cada elemento es
un tipo de dato distinto, pero para poder usar append todos deben ser
de la misma dimensión (mismo tipo de dato)
#x es un escalar, y0 es un arreglo de un número, B() devuelve un
arreglo de una lista
for i in range(n):
    dy0=A(x)@y0+B(y0[0])
    x+=h/2
    y1=y0+(h/2)*dy0

```



```

dy1=A(x)@y0+B(y1[0])

y2=y0+(h/2)*dy1
dy2=A(x)@y0+B(y2[0])

x+=h/2
y3=y0+h*dy2
dy3=A(x)@y0+B(y3[0])

y=y0+(h/6)*(dy0+2*dy1+2*dy2+dy3)
y0=y.copy()
R.append([x,y0[0][0],B(y0[0])[0][0]])#Esto forma un vector fila y
coloca cada valor nuevo en el último elemento
Res=np.vstack(R)#Forma una columna cada elemento que conformó la fila
con "append"
print(Res)
print('1-)Altura del líquido a los 60[s]:',Res[3][1])
print('2-)Volumen del líquido a los
60[s]:',(1/3)*np.pi*Res[4,1]**2*Res[4,1])#Tener en cuenta que de la
geometría aplicada el radio y nivel del líquido coinciden en valor
print('3-)El área de la superficie libre de liquido a los
100[s]:',np.pi*Res[5,1]**2)
print('4-)Tiempo en minutos en que se descarga el tanque:',1800/60)
print('5-)Rapidez con la que disminuye el nivel del liquido a los 20[s]
en [in/s]:',Res[1,2]*12)

```

```

1-)Altura del líquido a los 60[s]: 7.892691254544932
2-)Volumen del líquido a los 80[s]: 507.8138759404414
3-)El área de la superficie libre de liquido a los 100[s]: 192.11192582932838
4-)Tiempo en minutos en que se descarga el tanque: 30.0
5-)Rapidez con la que disminuye el nivel del liquido a los 20[s] en [in/s]: -0.02138864855338074

```

Segundo final 2024-02 Tema 3 (Simpson 1/3)

Consideraciones para la resolución de los temas:

- Utilizar seis decimales para las respuestas redondeando el ultimo decimal.
- Utilizar todos los decimales para los cálculos.
- No utilizar formato exponencial.
- No es necesario realizar ninguna conversión de unidades.

La masa total de una barra de densidad variable está dada por:

$$m = \int_0^L \rho(x) \cdot A(x) \cdot dx$$

x	0	1	2	3	4	5	6	7	8	9	10
ρ	2.3	2.43	2.6	3	3.4	3.5	3.8	4.2	4.38	4.8	5.7
A	2	3.24	2.3	1.8	1.68	1.3	1	0.86	0.75	0.7	0.66

Donde: m= masa, $\rho(x)$ = densidad, A(x)= Área de la sección transversal, x = distancia a lo largo de la barra y L= longitud total de la barra.

Determine utilizando el método de Simpson 1/3:

1. El volumen del cuerpo.
2. La masa del cuerpo.
3. La abscisa del centro geométrico del cuerpo.
4. La abscisa del centro de masa del cuerpo.

```
#2F 2024_02 T3 ( Simpson_tercio)
import numpy as np
import os
os.system('cls')
np.set_printoptions(suppress=True)
#Datos
x=np.array([0,1,2,3,4,5,6,7,8,9,10],dtype=float)
p=np.array([2.3,2.43,2.6,3,3.4,3.5,3.8,4.2,4.38,4.8,5.7],dtype=float)
S=np.array([2,3.24,2.3,1.8,1.68,1.3,1,0.86,0.75,0.7,0.66],dtype=float)
def simp(y):
    #####RUTINA015#####
    h=1#de la tabla de valores de x se ve que el paso es cte=1
    A=0
    k=0
    n=len(y)-1
    while(k<n):
        A+=(h/3)*(y[k]+4*y[k+1]+y[k+2])
        k+=2
    return A

print('1-)Volumen del cuerpo:',simp(S))#integral de Area*dx
print('2-)La masa del cuerpo:',simp(p*S))#integral de p*Area*dx
```

```
print('3-)Abscisa del centro geométrico:',simp(S*x)/simp(S))#integral(  
x*Area*dx )/volumen  
print('4-)Abscisa del centro de masa:',simp(p*S*x)/simp(p*S))#integral  
(densidad*Area*x*dx)/(masa total) (Momento/masa)
```

```
1-)Volumen del cuerpo: 15.240000000000002  
2-)La masa del cuerpo: 48.3656  
3-)Abscisa del centro geométrico: 3.566929133858267  
4-)Abscisa del centro de masa: 4.237066565217152
```

Segundo final 2025-01 Tema 1 (Trapecios)

Consideraciones a tener cuenta para la resolución de los temas:

- utilizar seis decimales para las respuestas redondeando el ultimo decimal
- utilizar todos los decimales para los cálculos
- no utilizar formato exponencial
- utilizar una tolerancia de 10^{-3} para los cálculos
- no es necesario realizar ninguna conversión de unidades

Tema 1

La masa total de una barra de densidad variable está dada por:

$$m = \int_0^L \rho(x) A_C(x) dx$$

El primer momento de la masa M_x de una barra de densidad variable está dada por:

$$M_x = \int_0^L x \rho(x) A_C(x) dx$$

El centro de masa en el eje x

$$\bar{x} = \frac{M_x}{m}$$

donde m = masa, $\rho(x)$ = densidad, $A_C(x)$ = área de la sección transversal, x = distancia a lo largo de la barra y L = longitud total de la barra (6 metros).

Se midieron los datos siguientes para una barra de 6 m de longitud. Determine la masa en kilogramos con la exactitud mejor posible.

x	0	1	2	3	4	5	6
ρ	2.32	2.42	2.63	3.1	3.43	3.83	4.21
A_C	2.2	3.23	2.31	2	1.69	1.31	1.2

Determine con ayuda del método del Trapecio:

a) la masa en kilogramos utilizando el m :

b) el volumen

c) el centro de masa en el eje x

```
#2F 2025_01 T3 ( Trapecio)
import numpy as np
import os
os.system('cls')
np.set_printoptions(suppress=True)
#Datos
x=np.array([0,1,2,3,4,5,6],dtype=float)
p=np.array([2.32,2.42,2.63,3.1,3.43,3.83,4.21],dtype=float)
S=np.array([2.2,3.23,2.31,2,1.69,1.31,1.2],dtype=float)
def Trapecio(y):
    #####RUTINA014#####
    h=1#se ve que los pasos son cte=1 en x
    A=0
    k=0
    n=len(x)-1
    while(k<n):
        A+=(h/2)*(y[k]+y[k+1])
        k+=1
    return A

print('a-)La masa del cuerpo:',Trapecio(p*S))#integral de p*Area*dx
```

```
print('b-)Volumen del cuerpo:',Trapecio(S))#integral de Area*dx
print('c-)Abscisa del centro de
masa:',Trapecio(p*S*x)/Trapecio(p*S))#integral
(densidad*Area*x*dx)/(masa total) (Momento/masa)
```

a-)La masa del cuerpo: 35.983900000000006

b-)Volumen del cuerpo: 12.239999999999998

c-)Abscisa del centro de masa: 2.834503764183426

Segundo final 2025-01 Tema 2 (Euler)

? **Tema 2** El movimiento de un péndulo balanceándose de acuerdo con ciertas suposiciones de simplificación se describe por medio de la ecuación diferencial de segundo orden

$$\frac{d^2\theta}{dt^2} + \frac{g}{L} \sin \theta = 0 ; \quad \theta(0) = \frac{\pi}{3} ; \quad \theta'(0) = 0,$$

La longitud de la cuerda L es de 3 pies de largo y que $g=32.24$ pies/s². Con $h=0.1$ s

Utilizando el método de EULER. Determinar: a) La posición angular del cuerpo en $t=0.2$ s ;
b) La velocidad angular del péndulo en $t=0.3$ s

```
#2F 2025_01 T2 ( Euler)
import numpy as np
import os
os.system('cls')
np.set_printoptions(suppress=True)
#Datos
L=3
g=32.24
h=0.1
y0=np.array([[np.pi/3],[0]])#y0=[posición angular, vel. angular]
x0=0
#Ec. diferencial: d2y/dy2=-g*np.sin(y)/L

#####RUTINA020##### Euler
m=len(y0)
a=np.hstack((np.zeros((m-1,1)),np.eye((m-1))))
A=lambda x:np.vstack((a,np.array([[0,0]],dtype=float)))
B=lambda y:np.vstack((np.zeros((m-1,1)),np.array([[-g*np.sin(y)/L]],dtype=float)))
x=x0
R=[]
R.append([x,y0[0,0],y0[1,0],(A(x)@y0+B(y0[0]))[1,0]])
n=5
for i in range(n):
    dy=A(x)@y0+B(y0[0])
    y=y0+h*dy
    y0=y.copy()
    x+=h
    R.append([x,y0[0,0],y0[1,0],(A(x)@y0+B(y0[0]))[1,0]])
Res=np.vstack(R)
print(Res)
print('a-)Posición angular del cuerpo en t=0.2[s]:',Res[2,1])
print('b-)La velocidad angular del péndulo en t=0.3[s]:',Res[3,2])

a-)Posición angular del cuerpo en t=0.2[s]: 0.95412868780323
b-)La velocidad angular del péndulo en t=0.3[s]: -2.7381012457183393
```

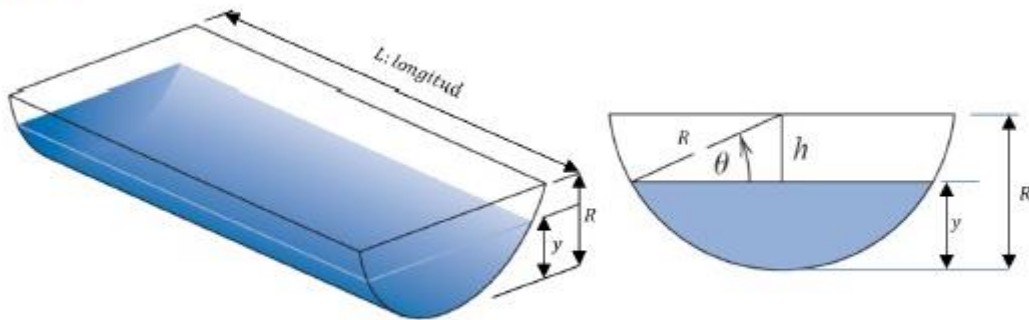
Tercer final 2025-01 Tema 3 (RKO4)

Consideraciones a tener cuenta para la resolución de los temas:

- utilizar seis decimales para las respuestas redondeando el ultimo decimal
- utilizar todos los decimales para los cálculos
- no utilizar formato exponencial
- utilizar una tolerancia de 10^{-2} para los cálculos
- no es necesario realizar ninguna conversión de unidades

Tema 3

Si se drena el agua desde un tanque semicilíndrico horizontal tal como se muestra en la figura por medio de una válvula en la base, el líquido fluirá rápido cuando el tanque esté lleno y despacio conforme se drene. Como se ve, la tasa o velocidad a la que el nivel (profundidad/altura y) del agua disminuye y es proporcional a la raíz cuadrada de la altura y del fluido dentro del tanque. Teniendo en cuenta que la constante de proporcionalidad es de $k = 0.6$, la profundidad del agua y se mide en $[m]$: metros, y el tiempo t : en minutos. No es necesario realizar conversión alguna para los cálculos. Sabiendo que el nivel del fluido inicialmente es de $1.4[m]$, y que el paso a ser utilizado es de $h = 0.1[min]$. El radio del cilindro es de $R = 2[m]$. La longitud del cilindro es de $1.2 [m]$.



Emplear el método de RKO^4 para obtener la función tabulada de $y = f(t)$.

Determinar:

1) Con la ayuda de la función $y = f(t)$, calcular el valor numérico esperado para:

1. El instante inmediatamente anterior al que el tanque se vacía.
2. Altura del nivel del líquido en $t=0.4$ segundos.
3. Caudal en $t=0.6$ segundos.
4. El valor del área de la superficie libre de líquido a los 0.7 segundos.
5. Rapidez con que baja el nivel del líquido a los 1 segundos.


```

#3F 2025_01 Tema 3 ( Rk4 )
import numpy as np
import os
os.system('cls')
np.set_printoptions(suppress=True)
#Datos
r=2#radio del cilindro
h=0.1#paso
k=0.6#cte de proporcionalidad
L=1.2#logitud del cilindro
y0=np.array([1.4])#y0=[nivel del líquido]
x0=0
def ancho(y):#función que devuelve el ancho de la superficie del
líquido de acuerdo a la altura
    return 2*(r**2-(y-r)**2)**0.5
#Ec. diferencial: dy/dt=-k*(y)**0.5
#####RUTINA023##### Rk4
m=len(y0)
a=np.hstack((np.zeros((m-1,1)),np.eye((m-1))))
A=lambda x:np.vstack((a,np.array([0]),dtype=float)))
B=lambda y:np.vstack((np.zeros((m-1,1)),np.array([-
k*(y)**0.5]),dtype=float)))
x=x0
R=[]
R.append([x,y0[0],(A(x)@y0+B(y0[0]))[0,0]])
n=50
for i in range(n):
    dy0=A(x)@y0+B(y0[0])
    x+=h/2
    y1=y0+(h/2)*dy0
    dy1=A(x)@y1+B(y1[0])
    y2=y0+(h/2)*dy1
    dy2=A(x)@y2+B(y2[0])
    x+=h/2
    y3=y0+h*dy2
    dy3=A(x)@y3+B(y3[0])
    y=y0+(h/6)*(dy0+2*dy1+2*dy2+dy3)
    y0=y.copy()
    R.append([x,y0[0,0],(A(x)@y0+B(y0[0]))[0,0]])
Res=np.vstack(R)
print(Res)
print('1-)Instante inmediato anterior al que el tanque s evacía:',3.8)
print('2-)Altura del nivel del líquido en t=0.4[s]:',Res[4,1])
print('3-)Caudal en t=0.6[s]:',Res[6,2]*L*ancho(Res[6,1]))
print('4-)Valor del ancho superficial a los 0.7[s]:',ancho(Res[7,1]))
print('5-)Rapidez con que baja el nivel del líquido a los
1[s]:',Res[10,2])

```

```

1-)Instante inmediato anterior al que el tanque s evacía: 3.8
2-)Altura del nivel del líquido en t=0.4[s]: 1.1304281739072937
3-)Caudal en t=0.6[s]: -2.5075244504981242
4-)Valor del ancho superficial a los 0.7[s]: 3.400885422344047
5-)Rapidez con que baja el nivel del líquido a los 1[s]: -0.529929577644667

```


Primer final 2025-02 Tema 2 (Hamming+Heun)

Consideraciones a tener cuenta para la resolución de los temas:

- * utilizar seis decimales para las respuestas redondeando el ultimo decimal
- * utilizar todos los decimales para los cálculos
- * no utilizar formato exponencial
- * no es necesario realizar ninguna conversión de unidades

Sea $B(t)$ el número de bacterias en millones en el tiempo t , medido en segundos. Si el índice de reproducción b es $b = b_0 \cdot (2 + \sin(\frac{\pi t}{3}))$ y el índice de mortalidad d es proporcional al tamaño de la población (debido a la superpoblación), entonces el índice de crecimiento de las bacterias está dado por la ecuación logística

$$\frac{dB(t)}{dt} = b \cdot B(t) - k[B(t)]^2$$

Donde $d = kB(t)$. Suponga que

$$B(0) = 50976 ,$$

$$b_0 = 0.029,$$

$$y \quad k = 1.4 \times 10^{-7}.$$

Utilizando el método de Hamming con un paso de $h = 1$ segundo y de Interpolación de Newton, responda el cuestionario:

(se debe utilizar HEUN para hallar los valores faltantes)

(para las respuestas utilice seis decimales redondeando el último dígito; use todos los decimales para los cálculos; no utilice formato exponencial)

Responda las siguientes preguntas.

1. El número de bacterias en millones a los 2 segundos:



The correct answer is: 58535.944878

2. El número de bacterias en millones a los 6 segundos:



The correct answer is: 68126.674991

3. El índice de crecimiento de las bacterias en $t = 3$ segundos:



The correct answer is: 3069.05721

4. El índice de crecimiento de las bacterias en $t = 5$ segundos:



The correct answer is: 1558.295414

5. Con ayuda de los valores obtenidos con HAMMING (considerar solo hasta $t=8$ segundos), estimar con la función interpoladora (METODO DE NEWTON) del número de bacterias en $t = 4.5$ segundos:



The correct answer is: 65180.3434

```

#1F 2025_02 T2 ( Hamming , Heun y int_newton)
import numpy as np
import os
os.system('cls')
np.set_printoptions(suppress=True)
#Datos
y0=np.array([50976],dtype=float)#y0=[población]
k=1.4*1e-7
h=1
x0=0
#Ec. diferencial:p'=0.029*( 2+np.sin(np.pi*t/3) )*p-k*p**2

#Heun para puntos necesarios en Hamming modificado
#####RUTINA023##### Heun
m=len(y0)
a=np.hstack((np.zeros((m-1,1)),np.eye((m-1))))
A=lambda x:np.vstack((a,np.array([0]),dtype=float)))
B=lambda x,p:np.vstack((np.zeros((m-1,1)),np.array([0.029*(
2+np.sin(np.pi*x/3) )*p-k*p**2]),dtype=float)))
x=x0
yn=[]
yn.append(y0)
R=[]
R.append([x,y0[0],(A(x)@y0+B(x,y0[0]))[0,0]])
n=3
for i in range(n):
    dy1=A(x)@y0+B(x,y0[0])
    y1=y0+h*dy1
    x+=h
    dy2=A(x)@y1+B(x,y1[0])
    y=y0+(h/2)*(dy1+dy2)
    y0=y.copy()
    yn.append(y0)
    R.append([x,y0[0,0],(A(x)@y0+B(x,y[0]))[0,0]])

#####RUTINA027##### Hamming
y0,y1,y2,y3=yn
x1=x0+h
x2=x1+h
x3=x2+h
n=5
for i in range(n):
    dy1=A(x1)@y1+B(x1,y1[0])
    dy2=A(x2)@y2+B(x2,y2[0])
    dy3=A(x3)@y3+B(x3,y3[0])
    p=y0+(4/3)*h*(2*dy1-dy2+2*dy3)
    x4=x3+h
    dy4=A(x4)@p+B(x4,p[0])# mod se debe evaluar en B en este caso
    y=(9*y3-y1)/8+(3/8)*h*(-dy2+2*dy3+dy4)

```

```

y0=y1.copy()
y1=y2.copy()
y2=y3.copy()
y3=y.copy()
x1=x2
x2=x3
x3=x4
p0=p.copy()
R.append([x4,y[0,0],(A(x4)*y+B(x4,y[0]))[0,0]])
Res=np.vstack(R)
print(Res)
print('1-)Número de bacterias a los 2[s]:',Res[2,1])
print('2-)Número de bacterias a los 6[s]:',Res[6,1])
print('3-)El índice de crecimiento en t=3[s]:',Res[3,2])
print('4-)El índice de crecimiento de bacterias en t=5[s]:',Res[5,2])

#####RUTINA013##### Interpolación de
Newton
x=Res[0:,0]
y=Res[:,1]
n=len(x)
p=np.poly1d([1,-x[0]])
P=np.poly1d(y[0])
for i in range(1,n):
    a=(y[i]-P(x[i]))/p(x[i])
    P+=a*p
    p=np.polymul(p,np.poly1d([1,-x[i]]))
print('5-)Número de bacterias a los 4.5[s]:',P(4.5))

```

```

1-)Número de bacterias a los 2[s]: 58535.94487838019
2-)Número de bacterias a los 6[s]: 68126.67499110861
3-)El índice de crecimiento en t=3[s]: 3069.0572096540063
4-)El índice de crecimiento de bacterias en t=5[s]: 1558.2954144702344
5-)Número de bacterias a los 4.5[s]: 65180.34340030221

```

Primer final 2025-02 Tema 3 (Trapecios+Falsa posición)

Consideraciones a tener cuenta para la resolución de los temas:

- utilizar seis decimales para las respuestas redondeando el ultimo decimal
- utilizar todos los decimales para los cálculos
- no utilizar formato exponencial
- no es necesario realizar ninguna conversión de unidades
- utilizar una tolerancia de 10^{-5} para los cálculos

Hallar el valor de k para que el área entre los puntos $[7,9]$ bajo la curva $5 * x^2 + (2 * k + 10) * x + 2 * k + 31$ valga 100 por el método del trapecio con $n=10$. Utilizar el método de falsa posición siendo los valores de a y b los enteros mas próximos a la raíz y la tolerancia $1e-5$. Determinar:

a) el valor de k :

✓

The correct answer is: -21.261111

b) con el valor determinado, calcular el Área:

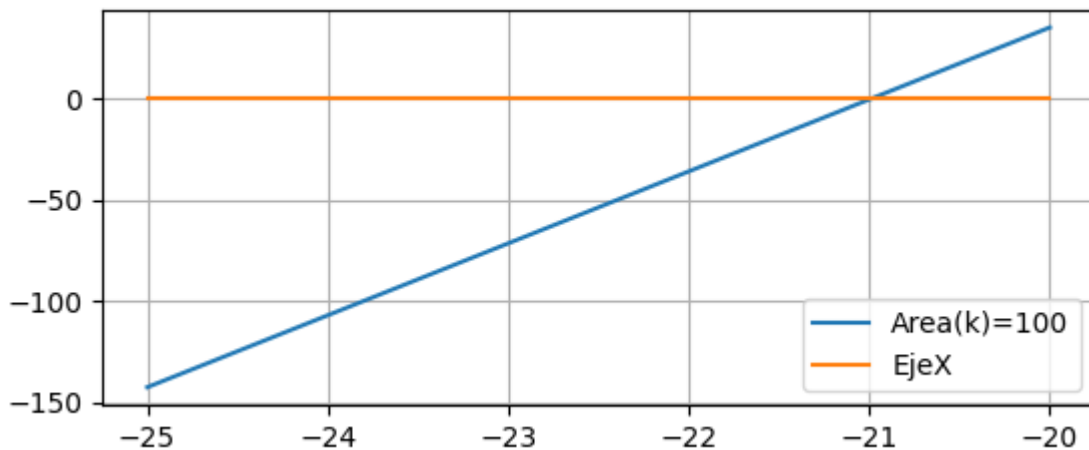
✓

The correct answer is: 100

c) Además de eso determinar el error cometido en el área al utilizar la tolerancia dada:

✗

The correct answer is: 0



```
#1F 2025_02 T3 ( Trapecios y Falsa Posición)
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import os
```

```
os.system('cls')
```

```
np.set_printoptions(suppress=True)
```

```
#Datos
```

```
ai,bi=(7,9)#para intervalo de integración
```

```
n=10
```

```
tol=1e-5
```

```
def F(x,k):
```

```
    return 5*x**2+(2*k+10)*x+2*k+31
```

```
def trapecio(f,x,a,b):
```

```

#####RUTINA014##### Integración por
trapecios
h=(b-a)/n
A=0
k=0
for i in np.arange(a,b,h):#defino la rutina tal que se pueda trabajar
con dos incógnitas en la integración
    A+=(h/2)*(f(i,x)+f(i,x))
return A
f=lambda k:trapecio(F,k,7,9)-100
xvals=np.linspace(-25,-20,40)
plt.plot(xvals,f(xvals),label="Area(k)=100")
plt.plot([-25,-20],[0,0],label="EjeX")
plt.legend()
plt.grid()
plt.show()
#####RUTINA005#####
a,b=(-22,-21)
c0=a
n=200
for i in range(n):
    c=(b*f(a)-a*f(b))/(f(a)-f(b))
    err=np.abs(c0-c)
    relerr=np.abs(err/c)
    if tol>err or tol>relerr or tol>np.abs(f(c)):
        break
    else:
        if f(a)*f(c)<0:
            b=c
        else:
            a=c
    c0=c
print('a-)El valor de k:',np.round(c,6))
print('b-)Valor del Área:',np.round(trapecio(F,c,7,9),6))
print('c-)Error cometido en el área:',np.round(np.abs(100-
trapecio(F,c,7,9)),6))

```

a-)El valor de k: -21.246067

b-)Valor del Área: 100.0

c-)Error cometido en el área: 0.0